

High-level Testkonzept INSIS Implementierung*

Version 1.0 vom 08.07.2021 13:54

*

Das Testkonzept zur Implementierung der In-/Exkasso- und Buchhaltungsfunktionen wird als Erweiterung dieses Konzeptes zu einem späteren Zeitpunkt hinzugefügt, ebenso für die Testbegleitung der Migration.

Inhaltsverzeichnis

1	Einführung.....	4
2	Ziele	5
2.1	Testziele.....	5
2.2	High-level Konzept.....	5
2.3	Zielgruppe	5
2.4	Gegenstand der Abgrenzung	6
2.5	Befüllung des Dokumentes.....	6
3	Testplanung und -steuerung.....	7
3.1	Testprozess	7
3.2	Fehlermanagementprozess	8
3.3	Change Request-Prozess.....	10
3.4	Zeitliche und organisatorische Planung.....	11
3.5	Rollen, Verantwortung und Kommunikation	14
3.6	Kommunikation und Meetings	16
3.7	Risikobewertung	17
3.8	Teststrategie	17
3.9	Teststufen	17
3.10	Teilweise automatisierbare Testarten.....	18
3.11	Nicht-funktionale Tests (NFT).....	19
3.12	Allgemeine Testtechniken.....	20
4	Testumgebungen	21
4.1	Berechtigungskonzept	21
4.2	Testumgebungsmanagement.....	21
4.3	Qualitätsmerkmale	23
4.4	Kriterien: Eingang, Ende, Abnahme, Abbruch, Wiederaufnahme	24
5	Testanalyse und -design	29
5.1	Testbasis.....	29
5.2	Qualitätssicherung der Testbasis	29
5.3	Schnittstellenintegration.....	29
5.4	Funktionale Workflows.....	29
5.5	Testfallermittlung und Testtiefe.....	31
5.6	Testobjekte	32
5.7	Erstellung der Testfälle	32
5.8	Vorbereitung der Tests	32
5.9	Auswahl der Testtools	33
5.10	Testdaten	34
6	Testrealisierung und -durchführung	36
6.1	Testdurchführung und Testabdeckung.....	36
6.2	Testautomatisierung	36
7	Testauswertung und -bericht.....	37
7.1	Testdokumentation	37
7.2	Testmetriken	37
7.3	Reporting	37

8	Testabschluss	38
8.1	Retrospektive	38
8.2	Lessons Learned	38
8.3	Optimierung des Testportfolios	38
9	Anhang	39
9.1	Zu erstellende Artefakte	39
9.2	Tabellen- und Abbildungsverzeichnis	39
9.3	Glossar	40
9.4	Referenzen	40
9.5	Versions-Historie	41
9.6	Abnahme des Dokuments	42
9.7	Informationen zum Dokument	42

1 Einführung

Die mainframebasierten Systeme des ADAC für Versicherungsbestand, -vertrag und Schadensregulierung haben das Ende ihres Lebenszyklus erreicht und sollen durch eine zukunftsfähige Lösung ersetzt werden, die folgende Kriterien erfüllt:

- Zukunftsfähige Technologie, dies auch mit Hinblick auf Digitalisierung
- Standard Software bzw. marktgängige Plattform
- Hoher Automatisierungsgrad
- Reduktion der Komplexität der implementierten Lösung
- Verbesserte Time-to-Market
- Reduktion der Kosten für kontinuierliche Verbesserung und Betrieb

Nach Abschluss des Ausschreibungsverfahrens und als Ergebnis eines Proof of Concepts fiel die Wahl auf die Standard Versicherungssuite INSIS der Firma Fadata. Das Umsetzungsprojekt ist Teil des Programms **VITA**.

Das Programm **VITA** deckt damit die Umsetzung eines zentralen Teils der Anforderungen aus der Business- und der IT-Plattform-Strategie für die ADAC Versicherung AG ab.

Die Hauptthematiken hierbei sind:

- Zwingende Grundvoraussetzung für die Digitalisierung des Versicherungsgeschäfts und ein verbessertes Kundenerlebnis
- EIN integriertes Versicherungssystem für Bestand und Schaden mit ADAC- und spartenübergreifendem Produktmodell
- Moderne Produktmaschine für vereinfachte und schnelle Produkteinführungen & -anpassungen
- End-to-End Digitalisierung der Produktmodellierung über Vertrag bis zur Regulierung & Omnikanalfähigkeit
- Flexible Einbindung neuer Funktionalitäten und schnelle Anbindung externer Partner und Dienstleister, z.B. BiPro
- Begleitung von Demografie und Fachkräftemangel durch weitere Automatisierung & Ausbau der Dunkelverarbeitung
- Verbesserte Datenqualität, Analytics-Fähigkeiten für das Reporting und kontinuierliche Produkt- & Prozessverbesserung

Die Implementierung von INSIS ist das größte Projekt des Programmes VITA. Die Umsetzung der In- und Exkasso-Funktion für VITA auf Basis von SAP FS-CD ist ein weiteres Projekt des Programms.

Dieses Dokument stellt die wesentlichen Grundzüge des Testvorgehens bei der INSIS Softwareimplementierung im Sinne einer high-level Teststrategie dar. Da im ADAC diese Inhalte unter dem Begriff Testkonzept verstanden werden, wird dieses Dokument im Folgenden high-level Testkonzept bezeichnet.

2 Ziele

Dieses Dokument bietet einen strategischen Rahmen und eine allgemeine Grobvorgabe für das Testen der Softwareprodukte des VITA Programms, basierend auf einem Best Practices Ansatz. Dies wird als Wegweiser für alle geplanten Programmphasen verwendet. Es stellt somit ein Masterkonzept, das welches pro Release und auch mit Hinblick auf weitere Projekte des Programmes (z.B. In-/Exkasso) konkretisiert und ergänzt wird.

2.1 Testziele

Das frühzeitige Erkennen von Fehlern in der Anforderungsdefinition und Softwareanpassung mitigiert die mit der Implementierung verbundenen Risiken und Kosten. Daher ist das Hauptziel der geplanten Testaktivitäten, möglichst früh Fehler in der Konfiguration, in der Programmierung und der Integration zu erkennen, um daraus Erkenntnisse zum Stand der Produktqualität ableiten zu können.

Neben der Sicherstellung der funktionalen Anforderungen werden auch einige nicht-funktionale Anforderungen, wie z.B. Performance getestet.

Für die vergleichende Messung des Qualitätsniveaus wird auf Basis von identifizierten und ausgewählten Testfällen ein Risiko-basiertes Regressionsportfolio aufgebaut, welches die Priorität und Kritikalität der Anforderungen widerspiegeln.

Da die zur Verfügung stehenden Testressourcen limitiert sind und ein zeitlich straffer Implementierungsplan vorgegeben ist, soll ein adäquater Automatisierungsgrad erreicht werden.

Ein weiteres Ziel ist es, die geplanten Testaktivitäten ressourcenschonend durchzuführen. Dafür werden eine geeignete Testarchitektur, Testinfrastruktur und ein effektives und effizientes Testumgebungsmanagement benötigt.

2.2 High-level Konzept

Das high-level Testkonzept skizziert die Leitplanken der anstehenden Testaktivitäten. Es enthält alle zum jetzigen Zeitpunkt bekannten, wesentlichen Informationen zum allgemeinen Testprozess, zur Testplanung, -durchführung, -auswertung und zum Post-Testing Review rund um die Implementierung von INSIS.

Das high-level Testkonzept dient als Leitfaden für das konkrete Projekt und die jeweiligen Projektphasen. Zweck des Testkonzeptes ist es, den Testansatz, die Testtypen, die Teststufen, das Testvorgehen, Testumgebungen und die Tools zu definieren.

Dieses Dokument wird in den nächsten Phasen ergänzt, wenn Anforderungen, Testumgebungen und ein Build-Management-Ansatz, etc. konkreter ausgearbeitet sind, bzw. wenn sich die Testvorgehensweise oder Rahmenbedingungen ändern und eine Anpassung erforderlich wird.

Für die folgenden drei Testbereiche wird die Teststrategie im Projektfortgang ausspezifiziert.

- Für den Bereich Migration werden die geplanten Testaktivitäten durch die Verantwortlichen für „Migration“ abgedeckt, die Arbeiten erfolgen dabei in enger Abstimmung mit dem ADAC.
- Die Tests aus den Bereichen Failover- und Recovery werden federführend durch die Verantwortlichen für Infrastruktur / Betrieb durchgeführt.
- Für die Durchführung der Sicherheitstests (Pen-Test) wird eine externe Firma beauftragt. Die Beauftragung wird auf Anfrage des Projektes durch die Abteilung IT-Sicherheit erteilt.

Dieses high-level Testkonzept wird nicht als operatives Arbeitsdokument verstanden. Basierend auf dem Testkonzept wird die weitere Ausarbeitung und Detaillierung für die operative Planung, Durchführung und der Abschluss der geplanten Testaktivitäten für die nachfolgenden Projektphasen in je einem Releasetestkonzept, einem Teststufenplan und, je nach Projektentscheidung, Testumgebungskonzept dokumentiert.

2.3 Zielgruppe

Dieses Dokument wendet sich an alle Personen, die an der Planung, Vorbereitung und/oder Durchführung des Projektes VITA beteiligt sind, wie z.B. Tester und Testmanager, Projektleiter und Projektmitarbeiter, AIS mit den Bereichen IT-Sicherheit, Architektur, Betrieb sowie die Fachbereiche, Betriebsrat, Revision, Datenschutz und IT-Compliance.

Deshalb wird das Dokument für alle Stakeholder abrufbar [hier](#) im zentralen Projekt SharePoint abgelegt.

2.4 Gegenstand der Abgrenzung

Bei dem high-level Testkonzept handelt es sich um keine detaillierte Anleitung hinsichtlich der konkreten Tests oder deren Ausarbeitung und Durchführung, sondern um eine Beschreibung der nach derzeitigem Stand am besten geeigneten Vorgehensweise zur erfolgreichen Begleitung und Testung der Einführung von INSIS.

Hier werden weder Testplan, Testfälle oder Release Pläne betrachtet, noch die Migrationstests. Diese werden in nachgelagerten Dokumenten gesondert adressiert. Die Strategie selbst fußt auf einer dynamischen Projekt Roadmap, so dass eine Änderung der Projekt Roadmap auch eine Änderung der Teststrategie nach sich ziehen kann.

Im ADAC sind Testprozesse nach ISTQB Standard dokumentiert und implementiert.

In diesem Dokument werden diese Prozesse ergänzt und, wenn sinnvoll, adaptiert.

2.5 Befüllung des Dokumentes

Das Dokument wird nach dem Review mit der Version 1.0 nicht eingefroren. Es wird weiterhin Veränderungen und Erweiterungen im Projektverlauf erfahren.

Die Versionierung ist am Ende des Dokuments zu finden.

3 Testplanung und -steuerung

3.1 Testprozess

3.1.1 Agiles Vorgehen

Das Projekt verwendet agile Praktiken wie sie aus dem SCRUM-Ansatz bekannt sind.

Zu Beginn eines Sprints wird das Sprint Backlog definiert und das Team arbeitet an Design, Entwicklung, Test und Dokumentation der Anforderungsimplementierung, welche am Ende des Sprints im Rahmen einer Demo den Stakeholdern vorgestellt werden (Scrum Artefakt = Review). Aktuell ist die Sprintdauer für das Projekt auf 3 Wochen festgelegt.

Alle Work Streams werden in SCRUM-Teams organisiert, welche sich die Arbeiten der Implementierungsplanung, Anforderungsanalyse, Design, Entwicklung, Test sowie Teammanagement aufteilen. Begleitet werden die Teams von je einem SCRUM-Master, der die Rahmenbedingungen für die agile Vorgehensweise schafft. Aus dem Fachbereich wird die Rolle des Product Owners bestimmt, der für das Backlog verantwortlich ist. SCRUM Master und Product Owner übernehmen als Teamunterstützungsleistung zusätzlich zu ihren originären Aufgaben auch koordinative Arbeiten sowie das Fortschrittreporting des Teams.

Details zu den Rollen und der Kommunikation finden sich in Abschnitt 3.5 Rollen, Verantwortung und Kommunikation.

3.1.2 Quality Gates (QG)

Während der Sprint-Entwicklung werden neben Entwicklertests auch die InSprint-Tests auf Basis der Akzeptanzkriterien „Definition of Done“ (DoD) durchgeführt, die in jeder Anforderung „User Story“ definiert sind. Wurde die Erfüllung der Akzeptanzkriterien bestätigt, ist das reguläre Quality Gate erreicht, und es kann die Übergabe in spätere Testphasen erfolgen.

Für das Projekt wurde ein hybrides Modell ausgewählt:

- für die In-Sprint Tests nach der agilen Anwendungsentwicklung, siehe LINK
- für die Out-Of-Sprint Tests nach der klassischen Anwendungsentwicklung, siehe LINK

Die vollständige Dokumentation ist im Application Lifecycle Management (ALM) [definiert](#).

Quality Gate	Phase	Zusammenfassung
Eingang InSprint	Dev	Erforderliche Eingangsaktivitäten für den Start des InSprint Testing sind durchgeführt, z.B. User Stories auf „Ready for Test“.
Ausgang InSprint	Dev	Alle relevanten Vorlaufaktivitäten für SIT abgeschlossen, Alle dafür geplanten Testaktivitäten durchgeführt. Die Akzeptanzkriterien in User Stories wurden erfolgreich bestätigt.
Eingang SIT	SIT	Erforderliche Eingangsaktivitäten für SIT sind durchgeführt. Das SIT Testing ist bereit zum Start.
Ausgang SIT	SIT	Geplante SIT Tests sind erfolgreich durchgeführt worden. Die Freigabe zum Beginn Abnahme (BZA) ist erteilt. Der Link zu dem Quality Gate Formular befindet sich hier .
Eingang UAT	UAT	Erforderliche Eingangsaktivitäten für UAT sind durchgeführt. User Acceptance Testing ist bereit zum Start.
Ausgang UAT	UAT	Die Abnahme (BZA) Test sind erfolgreich abgeschlossen, Die Produktions-Freigabe vom Auftraggeber vor Rollout/ Betriebsübergabe vom Projektleiter nach Rollout (BZP) ist erteilt. Der Link zu dem Quality Gate Formular befindet sich hier .
Eingang NFT Test	SIT / UAT /Pre Go-Live	Erforderlich Eingangsaktivitäten sind durchgeführt. Die nichtfunktionalen Tests können beginnen.
Ausgang NFT Test	SIT / UAT /Pre Go-Live	Nichtfunktionale Tests sind erfolgreich abgeschlossen.
Ready for Go Live	GoLive	Bereit für Go Live.

Tabelle 1: Quality Gates

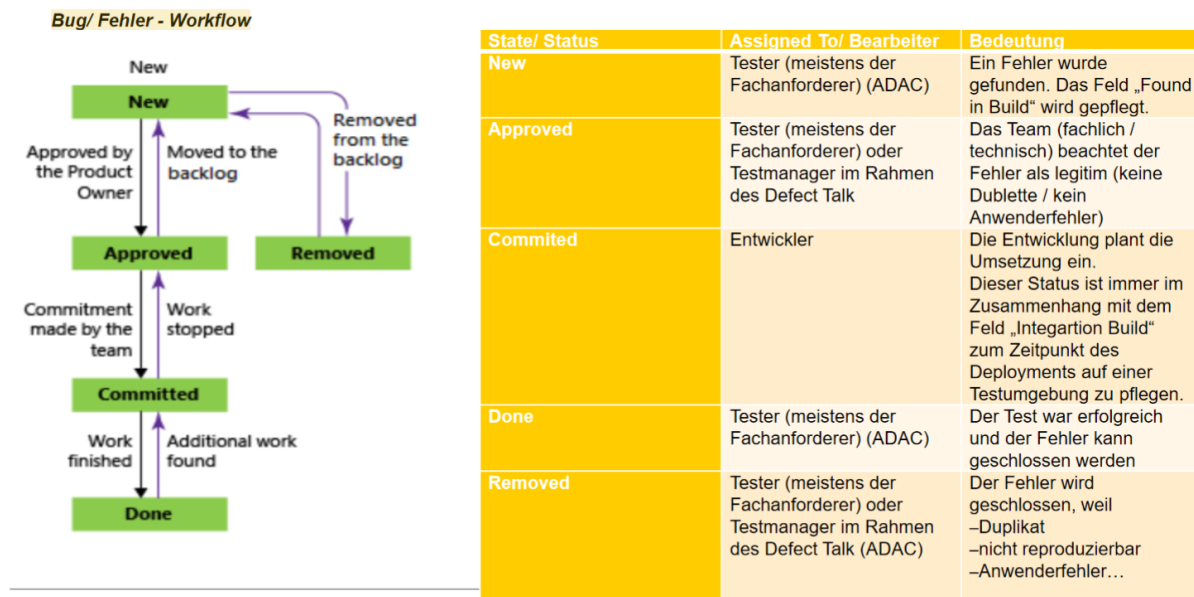
3.2 Fehlermanagementprozess

3.2.1 Beschreibung

Das Projekt folgt den allgemeinen Vorgaben des ADAC zum [Fehlermanagement \(Defect Management\)](#). Der Testmanager wird zusammen mit einem Defect Manager den Defect-Managementprozess im Projekt verantworten und darüber im Projektstatus und den Projekt- und Programmleitern berichten.

Im Projekt wird Azure DevOps von allen Projektteams (on-, near- und offshore) als zentrales Werkzeug für das Fehlermanagement genutzt, so dass alle Fehler (Schritte zur Reproduktion und Testergebnisse) dokumentiert werden und entsprechende Auswertungen mittels eines Dashboards für das Prozessmonitoring erstellt werden können. So ist eine Echtzeitauswertung der Fehlersituation und der Nachverfolgung einzelner Fehlerbehebungsschritte sowie eine Steuerung des Fehlerprozesses gewährleistet.

Der Defect-Workflow wird anbei exemplarisch beschrieben (zusätzliche Zwischenstatuswerte können von den Workstreams bei Bedarf ergänzt werden):

Abbildung 1: Defect Workflow Dokumentation (Scrum Template) aus Azure DevOps [LINK](#)

3.2.2 Testfall Priorisierung

Testfälle werden entsprechend der nachfolgenden Metrik anhand ihrer Priorität hinsichtlich der Auswirkungen auf den Geschäftsprozess kategorisiert. Eine weitere detaillierte Priorisierung kann anhand dieser [Vorlage](#) erfolgen.

Priorität	Beschreibung
1	Ein Testfall, der basale Schlüsselfunktion oder eine grundlegende Anforderung mit sehr hohem Fehlerpotential hinsichtlich geschäftskritischer Grundfunktionalitäten validiert und ein positives Szenario (erwartetes Verhalten der Anwendung) misst. Durch das Bestehen dieses Tests wird sichergestellt, dass das minimal erwartete Verhalten der Anwendung abgedeckt ist
2	Ein Testfall, der eine Schlüsselfunktion oder eine Anforderung mit hohem Risiko validiert und ein positives Szenario (erwartetes Verhalten der Anwendung) misst. Durch das Bestehen dieses Tests wird sichergestellt, dass das grundlegende erwartete Verhalten der Anwendung abgedeckt ist.
3	Ein Testfall, der eine Anforderung mit mittlerem Risiko, eine Komfort-Funktion (Benutzerfreundlichkeit) oder negative Szenarien, deren Auftreten unwahrscheinlich ist, validiert.

Priorität	Beschreibung
4	Ein Testfall, der eine Anforderung mit geringem Risiko, einen bestimmten Ablauf für verschiedene Datenkombinationen oder kleinere Anforderungen wie Schriftgröße, Position des Feldes, Ausrichtung usw. in Bezug auf die Benutzerfreundlichkeit validiert.

Tabelle 2: Testfall Priorisierungsmetrik

3.2.3 Bugs - Priorisierung & Klassifikation

Im Fehlermanagementprozess werden in allen Testphasen Defects (Bugs) durch die folgende Schweregradklassifizierung definiert:

Der Schweregrad eines Fehlers beschreibt damit das Ausmaß der Beeinträchtigung und die möglichen Folgen für den Anwender.

Schweregrad / Fehlerklasse	Beschreibung
1 Critical	Mängel der Fehlerklasse 1 führen dazu, • dass das System oder ein wesentlicher Teil nicht nutzbar ist ("die Anwendung steht") • oder der Betrieb in wesentlichen zeitkritischen Geschäftsprozessen unzumutbar behindert ist ("produktionsverhindernder Mangel") • oder dass eine wesentliche Funktion oder ein zeitkritischer Geschäftsprozess nicht ausgeführt werden kann oder fehlerhaft (nicht behebbare fachlicher Fehler) ist. Es kommt nicht zum Versagen des Systems insgesamt, ein Arbeiten ist mit starker Einschränkung in der Bedienung möglich. Der Mangel kann durch keinen fachlichen oder technischen Workaround umgangen werden oder der Workaround stellt für den Anwender einen erheblichen Mehraufwand dar, der ihm nur kurzfristig zugemutet werden kann.
2 High	Mängel der Fehlerklasse 2 führen dazu, dass innerhalb einer Funktion oder eines Geschäftsprozesses ein Fehler (korrigierbarer fachlicher Fehler) auftritt. Funktion oder Geschäftsprozess können aber trotz des Mangels mit kleinen Einschränkungen genutzt werden bzw. sind nicht so fehlerhaft, dass der Ablauf unzumutbar beeinträchtigt ist. Der Mangel kann durch einen Workaround umgangen werden, sofern der sich dadurch ergebende Aufwand für den Anwender zumutbar ist. Zumutbarkeit ist in diesem Fall gegeben, wenn die Nutzung verschoben werden kann bzw. eine Umgehung auch bei häufiger Nutzung nur geringen Mehraufwand verursacht.
3 Medium/	Mängel der Fehlerklasse 3 beeinflussen die Nutzung des Systems kaum. Die Abläufe von Funktionen und Geschäftsprozessen werden nicht merklich gestört. Der Mangel kann durch einen Workaround umgangen werden, der beim Auftraggeber zu einem nur unwesentlichen Mehraufwand führt. Das System erzielt trotz des Mangels stets ein richtiges Ergebnis (keine fachlichen Fehler). In der Datenbank werden keine fehlerhaften oder korrupten Daten abgespeichert. Eine zu starke Häufung von Fehlern der Klasse 3 für eine Anwendergruppe kann zu einer Beeinträchtigung der wahrgenommenen Systemqualität führen.
4 Low	Schönheitsfehler / Rechtschreibfehler, etc.

Tabelle 3 - Bugs, Priorisierung & Klassifikation

Die Priorität eines Fehlers die Dringlichkeit zu Behebung eines Fehlers.

Priorität	Beschreibung
1 Critical	Das Ereignis hat Auswirkungen auf die Funktionalität des gesamten Systems oder einer hochrangigen Komponente. Es gibt keinen Workaround für die Funktion (z. B. Absturz des Systems oder einer seiner Komponenten, Datenverlust, etc.). Der Test kann nicht fortgesetzt werden, da eine wichtige Funktion fehlt (Showstopper). Muss sofort behoben werden.
2 High	Das Ereignis hat Auswirkungen auf die Funktionalität einer hochrangigen Systemkomponente. Es gibt einen Workaround für diese Funktion (z. B. Betriebsfehler). Der Test für die Hauptfunktionalität kann aufgrund der fehlenden Funktionalität nicht wie geplant fortgesetzt werden, ein Workaround ist vorhanden. Muss sofort behoben werden.
3 Medium	Das Ereignis hat Auswirkungen auf die Funktionalität einer untergeordneten Komponente. Es kann einen Workaround für diese Funktion geben oder auch nicht. Der Test kann ohne Einschränkungen fortgesetzt werden. Muss so schnell wie möglich behoben werden.
4 Low	Das Ereignis hat keinen Einfluss auf die Funktionalität des Systems oder einer seiner Komponenten (z. B. UI-Layout). Hat keine Auswirkung auf den Test

Tabelle 4 - Priorität Bugs

3.3 Change Request-Prozess

Im agilen Projektvorgehen wird stets mit Anforderungsänderungen gerechnet. Diese sogenannten Change Requests (Änderungen der User Stories, Features oder Epics) werden vom Product Owner aufgenommen und priorisiert.

Abhängig von Art und Umfang der Veränderungen kann die Freigabe einer Änderung im Ermessen des SCRUM Teams erfolgen, wenn diese innerhalb des definierten Entscheidungsspielraumes liegt. Wird dieser überschritten, so sind weitere Change Request Prozesse wie z.B. Delegation an das INSIS Architekturremium, die Projektleitung, das AEB (Architekturentscheidungsboard) oder das Programm VITA erforderlich.

Die für den CR-Prozess benötigte Governance und die einzelnen Prozess-Schritte werden zu Beginn jeder Projektphase im Programm abgestimmt und sollte an dieser Stelle referenziert werden.

3.4 Zeitliche und organisatorische Planung

3.4.1 Zeitliche Planung Gesamtprojekt

Die Einführung von INSIS beim ADAC ist als Vier-Jahresprojekt geplant und sieht eine Laufzeit bis 2025 vor.

Für 2021 ist die Implementierung eines ersten Releases als MVP (Minimum Viable Product) geplant, das Ende Q1/2022 in Produktion ausgerollt werden soll.

Variante 1: Roadmap der zentralen Implementierungsobjekte

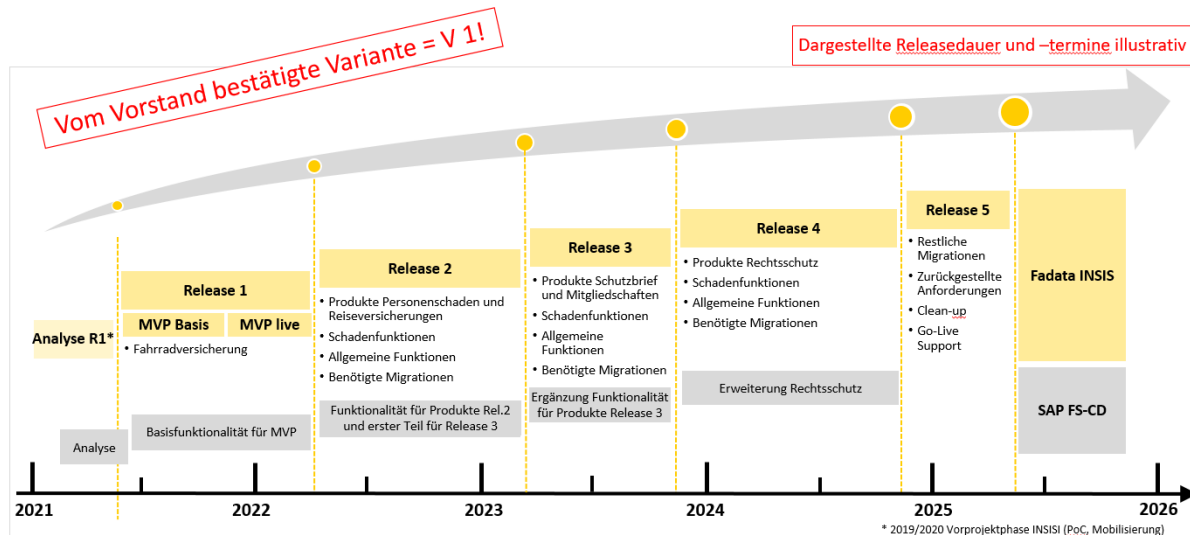


Abbildung 2: Grober Zeitplan für die Implementierung (Stand April 2021)

3.4.2 Meilensteinplanung pro Release

Grundsätzlich stellen der Anfang und das Ende jedes Releases wesentliche Meilensteine dar.

Weitere Meilensteine sind Anfang und Ende der Analysephase, der Implementierung (InSprint Test) sowie Anfang und Ende der jeweiligen Testphasen (InSprint, SIT, UAT, GoLive Tests und der Go-Live).

Eine Detaillierungsebene tiefer stellen sich Anfang und Ende eines jeden Sprints, sowie die jeweiligen Teststufen (InSprint,) als Meilenstein dar.

Sobald seitens Programm- /Projektleitung eine abgestimmte Gesamtplanung zur Verfügung steht, wird daraus ein Phasenplan mit Meilensteinen für Test abgeleitet und im Teststufenplan dokumentiert.

Planung INSIS Implementierung 1. Release MVP

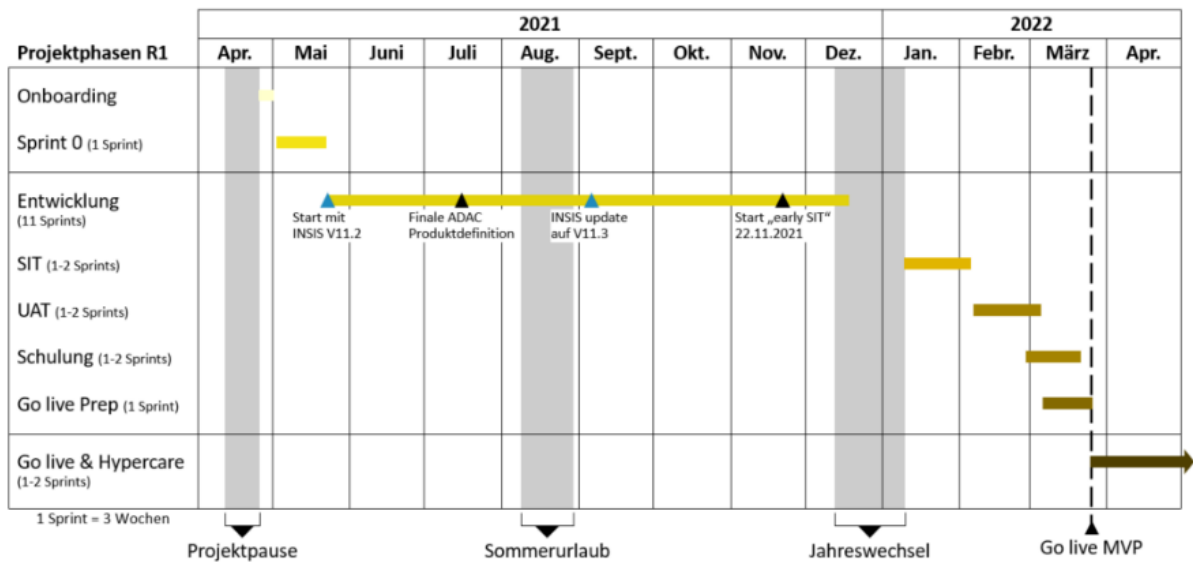


Abbildung 3: MVP Timeline (Stand April 2021)

3.4.3 Test Deliverables (Lieferprodukte) per Phase

Phase	Ergebnis	Beschreibung	Verantwortlich	Prüfer	Unterstützung
Analyse Phase	High-level Testkonzept	Beschreibt den high-level Ansatz zur Testmethodik, Testphasen, Umgebung, Testdatenanforderung, Tools, etc.	Projektleiter	PO und SME	Projektteam, Test-
InSprint Test	Testumgebung Readiness Report	Setup, Konfiguration und Deployment der Testumgebungen	Infrastruktur	Scrum Tester	
InSprint Test	Testausführung für InSprint-Testfälle	Test der User Stories während des InSprint-Tests	InSprint-Tester	PO, ADAC BA und SME	Scrum-BA
InSprint Test	Liveanzeige via Dashboard	Fortschritte bei der Testdurchführung InSprint	Scrum Tester	PO, Scrum Master, SME	Testmanager
InSprint Test	Wöchentlicher konsolidierter Testfortschrittsbericht	Fortschritt bei der Testdurchführung InSprint-pro Work Streams, Eskalation von kritischen Fehlern und Problemen, die mit den Testaktivitäten einhergehen.	Testmanager	PO und PL	Scrum Testers,
Ende InSprint Test	Sprintende Bericht	Sprintende-Bericht mit Angaben zum Status der User Stories (PBI) und zu ausstehenden Fehlern und ggf. zum Produktrisiko.	Testmanager	PO und PL	Scrum Testers, Scrum Masters

Phase	Ergebnis	Beschreibung	Verantwortlich	Prüfer	Unterstützung
SIT	Testausführung für SIT-Testfälle	Schnittstellentest von und zu INSIS. Tests der wesentlichen funktionalen Zusammenhänge aus technischer Sicht und aus Business-Sicht.	SIT Tester	ADAC BA, PO	Testmanager, Scrum BAs, SMEs
SIT	Liveanzeige via Dashboard	Täglicher Fortschritt bei der Testdurchführung.	Testmanager		
Ende SIT	Testabschlussbericht LINK	Testabschlussbericht mit detaillierten Angaben zum Status der Testausführung, zum Fehlerstatus, zu ausstehenden Fehlern und Risiken sowie zur Empfehlung der nächsten Phase.	Testmanager	PO	
Prozess-tests	Testausführung für Prozessfälle	Detaillierte Testszenarien und Testfälle aus Geschäftsprozessen erstellen sowie deren Ausführung	ADAC BA		Testmanager, BAs und SMEs
Prozess-tests	Liveanzeige via Dashboard	Täglicher Fortschritt bei der Testdurchführung.	Testmanager		
ENDE Prozess-tests	Testabschlussbericht LINK	Testabschlussbericht mit detaillierten Angaben zum Status der Testausführung, zum Fehlerstatus, zu ausstehenden Fehlern und Risiken sowie zur Empfehlung der nächsten Phase.	Testmanager	PO	
UAT	Testausführung der UAT Test-szenarien und Testfälle	Detaillierte Testszenarien und Testfälle, die aus den Geschäftsprozessen im Endzustand (Gesamtlebenslauf resp. End to End) erstellt und ausgeführt werden	UAT Tester / Testmanager	ADAC BA, PO	
UAT	Täglicher und wöchentlicher Testfortschrittsbericht	Täglicher Fortschritt bei der Testdurchführung	Testmanager		
Ende UAT	Testabschlussbericht	Testabschlussbericht mit detaillierten Angaben zum Status der Testausführung, zum Fehlerstatus, zu ausstehenden Fehlern und Risiken sowie zur Empfehlung der GoLive Phase	Testmanager	PO	

Phase	Ergebnis	Beschreibung	Verantwortlich	Prüfer	Unterstützung
Nicht funktionaler Test	Nicht funktionaler Test, Test-szenarien und -fälle	Detaillierte Testszenarien und Testfälle aus nicht-funktionalen Anforderungen erstellen und durchführen	Nicht funktionaler Tester		Testmanager
NFT	Liveanzeige via Dashboard	Täglicher Fortschritt bei der Testdurchführung	Testmanager		
Ende NFT	Testabschlussbericht LINK	Testabschlussbericht mit detaillierten Angaben zum Status der Testausführung, zum Fehlerstatus, zu ausstehenden Fehlern und Risiken sowie zur Empfehlung der nächsten Phase.	Testmanager		PO

Tabelle 5 : Deliverables per Phase

3.5 Rollen, Verantwortung und Kommunikation

3.5.1 Rollen im Workstream Test

Rolle	Aufgabe
Scrum Business Analyst (Scrum BA)	Als Experte für INSIS unterstützt er den PO bei der Analyse und Formulierung der Anforderungen in Form von Epics, Features und User Stories. Er steht dem Testteam als Ansprechpartner und Fachberater zur Verfügung.
ADAC Business Analyst (ADAC BA)	Nimmt bereits im Rahmen der InSprint Tests gemeinsam mit dem PO an Reviews teil und arbeitet im Testsystem, um Knowhow aufzubauen und bei fachlichen Rückfragen den PO zu unterstützen. Übernimmt mit zunehmendem Testfortschritt weitere Testaufgaben.
Testanalyst (optional)	Unterstützt die methodische Testfallerstellung und Design, kann auch optional an der Testdurchführung beteiligt sein.
Product Owner (PO)	Ist verantwortlich für die Abnahme der Artefakte (Auf „Done“ setzen) in Azure DevOps. Er ist an der Testfallerstellung beteiligt, sowie der finale Entscheider bei der Defect Triage und verantwortet die Produktionsfreigabe mit. Ist an neuralgischen Punkten für die Entwicklung des Testmanagements sowie in der Abnahme des Produktes an definierten Punkten mitverantwortlich.
SCRUM Master	Steht dem Team als Moderator und Vermittler zur Verfügung und ist erste Instanz zur Eskalation für das Testteam.
Testmanager (TM)	Verantwortet vollumfänglich das Testmanagement des Projektes
Testkoordinator (TC)	Koordiniert auf operativer Ebene, die für eine Testphase geplanten Testaktivitäten, unterstützt den Testmanager und liefert Input zu Fortschritten und Ergebnissen
Defect Manager (optional)	Auch Fehlermanager, ggf. in Personalunion mit Testmanager, verantwortet den Prozess rund um Fehler und deren adäquate Handhabung.
Tester	Arbeitet an den im Testplan vorgesehenen Testfällen und stellt Abweichungen zwischen SOLL und IST fest. Er oder sie ist zuständig für die Durchführung der (Re-)Tests und die Dokumentation der Testergebnisse.
Entwickler	Erstellt das technische Lösungskonzept, implementiert die priorisierten User Stories und nimmt einen Entwicklertest vor. Ferner analysiert, kategorisiert und behebt er oder sie Fehler und gibt sie zum Retest frei.

Rolle	Aufgabe
Testautomatisierer	Verantwortet das Schreiben (inkl. Testautomatisierungskonzept), Warten und Ausführen von automatisierten Testfällen. Er oder sie berichtet den Automatisierungsfortschritt und dokumentiert die Ausführungsergebnisse.
Umgebungsmanager	Ist koordinativ (nicht umsetzend) zuständig für die Installation, den Betrieb und die Wartung der Testumgebung und der darauf befindlichen Instanzen. Zudem koordiniert er die Nutzung der Testumgebungen innerhalb (Workstream) und außerhalb (Projektübergreifend) des Projekts. Er oder sie sorgt ggf. für die Einrichtung der Testwerkzeuge und arbeitet in Abstimmung mit dem Testmanager
Testdatenmanager	Sorgt für die qualitative und quantitative Verfügbarkeit der richtigen Testdaten auf der richtigen Instanz zum richtigen Zeitpunkt. Er oder sie stimmt sich mit anderen Teilen des Projektes und den Umsystemen bzgl. eventueller Datenflüsse (Schnittstellen) und anderes ab. Arbeitet in Abstimmung mit dem Testmanager. Außerdem ist er für die Projektübergreifende Koordination der Testdaten verantwortlich (ADAM 2.0).

Tabelle 6: Rollen im Projekt 2

Jedes SCRUM Team hat mindestens einen Tester (variiert je nach Teststufe).

Die meisten koordinativen Testrollen agieren jedoch SCRUM Team-übergreifend, z.B. der Testmanager, Testkoordinator, Defect Manager etc.

Zudem können manche Rollen temporär in Personalunion gelebt werden und bei größerem Arbeitsaufkommen wieder auf mehrere Personen verteilt werden (z.B. Defect-Manager).

Die genaue Testteamzusammensetzung wird in den nachfolgenden Projektphasen im Rahmen des Release-Testkonzeptes und der Teststufenplänen detailliert.

3.5.2 Verantwortlichkeit von ADAC und Partnern

Die grobe Verteilung der Aufwände auf den ADAC und die Umsetzungspartner wurde anhand der Teststufen, Testarten sowie Rollen im Workstream Test erarbeitet.

Die Gesamtverantwortung für das Testmanagement des Projektes liegt beim Testmanager des ADAC. Er wird seitens der INSIS Implementierungspartner durch die Scrum Master (InSprint Tests) und für die SIT-, NFT und bei Bedarf auch UAT-Tests durch einen Testkoordinator unterstützt. Seitens ADAC arbeiten ihm Defect- Umgebungs- und Testdatenmanager zu.

Die Planung, Durchführung und Reporting der InSprint Tests erfolgt durch die Scrum Teams, wo ADAC und INSIS Partner Kollegen zusammenarbeiten. Neben dem Tester sind die Rollen des POs und Scrum Masters wesentlich für eine gute und zügige Durchführung der Testarbeiten, Fehlerpriorisierung und Bestätigung der DoDs.

Zusätzlich zu den funktionalen Tests der Anforderungen im Rahmen der SCRUM Teams werden seitens der ADAC Business Analysten weitere Testaktivitäten geplant, um seitens ADAC frühzeitig die User Stories kennen zu lernen und funktionale Tests für den User Acceptance Test zu entwickeln. Diese Testaktivitäten werden ausschließlich durch ADAC Kollegen durchgeführt. Ansprechpartner seitens des Implementierungspartners sind je nach Teststufe die SCRUM Master oder der Testkoordinator.

Neben der Erstellung und Durchführung von manuellen Tests sollen auch für INSIS automatische Testfälle erstellt werden. Der Aufbau des Testframeworks, die Automatisierung und Durchführung wird vornehmlich durch INSIS Partner-Kollegen jedoch in Abstimmung mit dem Testmanager und bei Bedarf auch mit den ADAC Legacy und TAU-Kollegen durchgeführt.

Für die SIT Phase ist die enge Zusammenarbeit von INSIS Partner und ADAC Testern erforderlich, da hier der Fokus auf das Testen der Schnittstellenkommunikation von INSIS mit weiteren ADAC Systemen liegt. Eine wichtige Aufgabe ist hier auch die enge Abstimmung mit dem Testumgebungs- und Testdatenmanager da hier Deployments, Umgebungskonfiguration mehrerer Systeme und Datenbereitstellung gut abgestimmt sein müssen.

Die spezifischen Tests für nicht-funktionale Anforderungen an INSIS werden teils durch die INSIS Partner und teils durch ADAC durchgeführt. Der Testumgebungs- und Testdatenmanager sowie das Infrastrukturteam unterstützen bei dem sachgerechten Aufbau und Instrumentalisierung der benötigten Testumgebungen.

Die Planung, Durchführung und Berichterstellung für den UAT obliegt den ADAC Kollegen, wobei bei Bedarf INSIS Partner aus den Scrum Teams unterstützen können.

Die Security Tests werden durch ein externes Unternehmen (beauftragt durch ADAC IT Security) in enger Abstimmung mit dem ADAC Testmanager durchgeführt, welcher für die Planung, Kommunikation und Bereitstellung einer geeigneten Umgebung sorgt.

In Vorbereitung des Betriebs werden Failover-/Recovery-Test durchgeführt, welche in enger Abstimmung von INSIS Partner Infrastrukturprovider und ADAC Betriebskollegen stattfinden.

Die Testarbeiten im Rahmen der Migrationsarbeiten werden seitens der INSIS Partner in Zusammenarbeit mit den ADAC Legacy Kollegen im Fachbereich, Betrieb und im Test durchgeführt, um die Vollständigkeit, Richtigkeit und Laufzeitanforderungen bestätigen zu können.

Im Zuge der Vorbereitung und Durchführung der Go-live Aktivitäten arbeiten ebenfalls das INSIS Partner Infrastrukturteam gemeinsam mit dem ADAC Testmanager, Fachbereich und Betrieb, um den Produktivgang vorab zu simulieren und dann am geplanten Tag erfolgreich durchzuführen.

Im Zuge der Analysephase wurde als Diskussionsgrundlage eine grobe Übersicht über die Aufwandsverteilung erstellt, welche im Zuge der Implementierungsphase gemäß den Projekterfordernissen adaptiert wird. Diese ist [hier](#) zu finden.

Test organisation									
	INSIS Partner							80% 20%	ADAC
In-sprint Test	Scrum Team (exkl. PO und SMEs)					INSIS Test Koordinator	ADAC BA		
	10%	90%							
Insprint-ergänzende Test mit Fokus auf Prozesse & Lernen	INSIS Test Koordinator	ADAC BA ADAC Testmanager							
					50%	50%			
SIT	WS INT Tester + SIT Tester				INSIS Test Koordinator	ADAC Testmanager	Legacy Tester	Defect Manager	
								80% 20%	
Testautomatisation	Automatisierungstester + Support Scrum Team					INSIS Test Koordinator	ADAC BAs		Legacy Tester + TAU-Experte
						50%	50%		
Testdaten/Testumgebung	Testdaten: Scrumteam	Testdaten: Automat.-T.	Testdaten: Anonym.	T.-Umgebung: Infra.-Team	INSIS Test Koordinator	ADAC Testmanager	T.-Daten: Legacy. T.-T.	T.-Daten: Anonym.	T.-Umgebung: ADAC Betrieb
							70%	30%	
Last- u. Performancetests (LuP)	Performancetest-Tester + Support Automatisierungstester				Infra.-Team	INSIS Test Koordinator	ADAC Fachbereiche	ADAC Architekt	ADAC Betrieb + LuP Experte
						70%		20%	10%
Security Tests (SEC)	Security-Testteam: Beauftragung eines externen Dienstleisters						Infra.-Team	INSIS Test Koordinator	ADAC IT-Sicherheit + Betrieb
							70%	30%	
Failover/Recovery Test						Infra.-Team	ADAC Betrieb		
			30%	70%					
UAT	Scrum Team Support		Testkoord. onsite	ADAC Defect Manager	ADAC BAs	ADAC Testteam	ADAC Fachbereiche		
Defectmanagement*	"Scrum M." für Triage	Testmanager offsite			T. Koord. onsite	"PO" für Triage	ADAC Defect Manager		
*Fehler- und Abweichungsmanagement									
Testmanagement	"Scrum Master"	Testmanager offsite			T. Koord. onsite	"PO"	ADAC Testmanager		
Go live	Infra.-Team				T. Koord. onsite	ADAC Testmanager	ADAC Betrieb und ADAC Fachbereiche		

Abbildung 4 Initial vermutete Aufwandsverteilung der Testrollen, die im Laufe der Releases ggf. angepasst wird (Stand 16.03.2021)

3.6 Kommunikation und Meetings

3.6.1 Kommunikation

Der Austausch von Dokumenten aller Art erfolgt über den VITA Sharepoint. Dort befindet sich sowohl der Test-Ordner als auch andere relevante Projektdokumente, wie z.B. das Projektlog und die Unterlagen der Statusmeetings. Die Dokumentation der Kommunikation bezüglich einzelner User Stories oder Testfällen findet in Azure DevOps über die Kommentarfunktion bzw. das Diskussionsfeld statt.

Informationen und organisatorisch-terminliche Abstimmungen finden über Telefon, Videokonferenztool, (bevorzugt MS Teams), den SharePoint und E-Mail statt, inkl. Verteiler. Diese werden nach entsprechenden Notwendigkeiten erstellt.

3.6.2 Meetings

Aufgrund der Rahmenbedingungen der derzeitigen Pandemie, finden keine Präsenzmeetings, sondern Online-Meetings statt. Bevorzugt werden dabei die im Hause ADAC standardmäßigen Videokonferenztools (bevorzugt MS Teams) Anwendung finden.

Innerhalb eines SCRUM Teams tauschen sich die Mitglieder täglich im jeweiligen Daily aus. Auf SCRUM Teamebene findet vier Mal pro Woche ein SCRUM of SCRUM Meeting statt, um Workstream-übergreifende Themen auszutauschen und zu adressieren.

Projektstatusmeetings finden wöchentlich statt, in denen ein Vertreter jedes Work Streams über Fortschritt, Planung und Themen des Projektlogs berichtet.

Für die späteren Testphasen werden seitens des Testmanagers weitere Statusmeetings, Defect-Triage und Freigabesitzungen organisiert. Die Häufigkeit und Teilnehmerkreis wird auf die aktuellen Projekterfordernisse abgestimmt.

3.7 Risikobewertung

Projektrisiken, die das Erreichen der Testziele gefährden, werden zentral im Projektlog gepflegt und nachverfolgt, siehe ProjectLog im [Projektlog Sharepoint VITA](#). Die Minderung der Projekt- und Produkt Risiken als solche obliegen der Programm- und Projektleitung. Dabei ist festzuhalten, dass nur Produkt Risiken durch den Test Work Stream gemanagt werden, nicht jedoch Projektrisiken.

Dem Testteam obliegt die Klassifizierung von Fehlern im Rahmen des Defect Managements. Im Zuge der Defect-Triage wird anhand der definierten Kritikalitätsstufen die Schwere und Bedeutsamkeit für den Betrieb definiert und bei entsprechender Schwere (Showstopper) gemeldet, sodass notwendige Entscheidungen der Programm- und Projektleitung, wie z.B. Freigaben, getroffen werden können. Der aktuelle Teststand ist dabei tagesaktuell via Azure DevOps über entsprechend einzurichtende Azure DevOps-Dashboards einzusehen.

3.8 Teststrategie

Als Teststrategieansatz für Prozess- und UAT Tests wird eine Kombination aus anforderungs- / anwendungsfall-basiertem und risikobasiertem Testen gewählt.

Risikobasiertes Testen nutzt das Wissen über die Risiken dazu, dass unter gegebenen Rahmenbedingungen (Aufwand, Zeit, Verfügbarkeit usw.) die Risiken möglichst minimiert werden. Je nach zu erwartendem Risiko und der Schwere der Auswirkungen beim Eintreten einer Fehlerwirkung sind die Testaktivitäten auf einzelne Systemteile zu verteilen. Kritische Systemteile bedürfen einer eingehenderen Betrachtung und stehen bei der Testdurchführung mit an erster Stelle.

Anforderungsgetriebenes Testen stellt die Erfüllung der Anforderungen an das System in den Vordergrund. Für User Stories ist die "Definition von Done" die Basis für die Anforderungsabdeckung. Für den User Acceptance Test kann für die Testfallerstellung vor allem auf die „anwendungsbasierte Testfallermittlung“ zurückgegriffen werden.

3.9 Teststufen

Der Weg zu jedem Release führt über vier wesentliche Teststufen und einige ergänzende Tests.

3.9.1 InSprint-Test

Bei den InSprint-Tests werden verschiedene Testaktivitäten durchgeführt, die den Unit- und Systemtests in traditionellen Liefermodellen ähnlich sind. Sie dienen der Überprüfung der Implementierungsarbeiten und der definierten Akzeptanzkriterien in User Stories. Dem InSprint Testing geht ein Entwickler-test (UNIT Test) voraus, um die Implementierungsarbeit bereits in der Entwicklungsumgebung zu prüfen. Die InSprint Tests zielen auf Prüfung der definierten User Stories ab, welche für die Konfigurationen und das Customizing von INSIS erstellt wurden. Ein expliziter Test von OOTB-Funktionalitäten ohne Konfigurations- oder Entwicklungsbedarf ist nicht vorgesehen, jedoch werden zumeist die wesentlichen Funktionen bereits implizit bei der Testdurchführung anderer Test mitgetestet.

Im InSprint Test werden vornehmlich dynamische, strukturierte Tests durchgeführt und deren Durchführung in Azure DevOps dokumentiert wird. Das Schreiben eines gesondert ausformulierten Testfalls ist nicht zwingend notwendig. Aus Reportinggründen wird jedoch zu jedem PBI einen Testfall erstellt,

welcher auch nur die Akzeptanzkriterien beinhalten kann. Auf der Ebene des InSprint Testing ist es zulässig mit Mocks zu arbeiten, um die einzelnen User Stories ausreichend testen zu können.

3.9.2 Systemintegrationstests (SIT)

Nach erfolgreichem Durchlaufen des InSprint Tests wird das Produkt in dieser Phase den Systemintegrationstests unterzogen. Im Fokus stehen hier die Tests der Interoperabilität von integrierten Systemen. Hier wird explizit gegen die Akzeptanzkriterien der Schnittstellendefinition sowie auf die fachliche Richtigkeit in einer integrierten Testumgebung geprüft. Fokus ist die Schnittstelle zwischen den integrierten Systemen, nicht die Prüfung des end 2 end (e2e) Prozesses.

3.9.3 Prozesstests

Das Ziel dieser Testphase ist das Sicherstellen der Funktionalität von Konfiguration, Customizing und Integration entlang definierter Sollprozesse. Hier wird nicht mehr mit an User Stories geknüpften Testfällen gearbeitet, sondern mit Testfällen, die ein Überprüfen der definierten Prozesse und Prozessabläufe sicherstellen sollen. Diese Prozesstests können bereits zeitnah zu den SITs beginnen, sobald das korrekte Funktionieren der beteiligten Schnittstellen im SIT bestätigt wurde. Zusätzlich zu den strukturierten Tests ist auch mit explorativen Tests v.a. durch die Key-User und SMEs zu rechnen.

3.9.4 User Acceptance Tests (UAT)

Nach erfolgreichem Durchlaufen der Systemintegrationstests wird das Produkt in dieser Phase den User Acceptance Tests unterzogen.

In dieser Teststufe wird Fokus auf das Testen von Prozessen und Prozessketten aus Anwendersicht gelegt - insbesondere auch „Lebenslauf-tests“ innerhalb des Geschäftsprozesses unter Berücksichtigung aller Integrationspunkte mit anderen Systemen. Es handelt sich dabei um sog. e2e Tests.

In der UAT-Phase wird nicht mehr mit an User Stories geknüpften Testfällen gearbeitet, sondern mit Testfällen, die ein Überprüfen der definierten Prozesse und Prozessabläufe sicherstellen sollen. Zusätzlich zu den strukturierten Tests ist mit einer Vielzahl an explorativen Tests v.a. durch die Key-User und SMEs zu rechnen.

Am Ende der UAT Tests steht die Freigabe für die Produktivsetzung respektive das Deployment auf die Produktivumgebung des Systems.

3.9.5 PreGolive Testing

Im Wesentlichen handelt es sich dabei um fachliche Abnahme Tests (Reihe von Regressionstestfällen), die durch den Fachbereich auf der produktiven Umgebung durchgeführt werden. Dies erfolgt auf der Produktivumgebung, jedoch vor der Produktivsetzung der Software – soweit dies aus Revisionsicht zulässig ist.

3.10 Teilweise automatisierbare Testarten

3.10.1 Smoke-Tests

Smoke-Tests stellen eine definierte Anzahl an kurzen und robusten Testfällen dar, um im initialen Test z.B. bei einer neuen Softwareversionsbereitstellung und/oder neuen Umgebung das Funktionieren von wesentlichen Anwendungsfällen zu testen. Diese sollen i.d.R. innerhalb kurzer Zeit und mit wenig Aufwand durchgeführt werden und wenn möglich als Teil eines Regressionsportfolios automatisiert werden, sodass sie in die Continuous Integration/Conditional Deployment (CI/CD) Pipeline aufgenommen werden können.

Smoke-Tests werden sollten nach jeder Bereitstellung als Teil des automatisierten Deployment-Mechanismus durchgeführt. Wo eine Integration in das Azure DevOps Deployment nicht möglich ist, werden manuelle Smoke-Tests durchgeführt.

3.10.2 Regressionstests

Unter Regressionstests versteht man die Wiederholung von Testfällen, um sicherzustellen, dass Modifikationen in bereits getesteten Teilen der Software keine neuen Fehler („Regressionen“) verursachen. Grundsätzlich wird vorgeschlagen, den Regressionstest auf jeder Teststufe durchzuführen, und das Regressionstestportfolio an die Testziele der jeweiligen Umgebung anzupassen.

Aufgrund des Wiederholungscharakters und der Häufigkeit dieser Wiederholungen ist es sinnvoll, dass insbesondere für Regressionstests Testautomatisierung zum Einsatz kommt.

Hierzu wird im weiteren Verlauf des Projekts entschieden, welche der Testfälle für eine Automatisierung geeignet sind.

3.11 Nicht-funktionale Tests (NFT)

Nichtfunktionale Tests dienen der Aufdeckung von u.a. Fehlverhalten bezüglich Performanz, allgemeiner Benutzbarkeit und Sicherheit. Es werden Last- und Performancetest (LuP) im Rahmen des Workstreams Test durchgeführt. Sicherheits-, Failover/Recovery und Migrationstests werden von anderen Workstreams verantwortet und durchgeführt.

3.11.1 Last und Performance Testing

Unter einem Last- und Performance Test versteht man einen Softwaretest, der eine in der Regel sehr hohe Last auf dem zu testenden System erzeugt und dessen Verhalten wie z. B. geforderte Antwortzeiten sowie Mengenverarbeitungen für den Produktivbetrieb untersucht. Dazu kann eine Simulation eingesetzt werden (aktuell ist im Hause ADAC das Tool NeoLoad dazu im Einsatz).

Die genaue Ausarbeitung des Vorgehens für die Last und Performance Tests wird im Release Testkonzept zu finden sein.

Der Link zu den Vorlagen zum LuP Testkonzept und LuP Testabschlussbericht befinden sich [hier](#).

Hierzu werden verschiedene Auslastungsszenarien genutzt, in welchen die definierten Geschäftsprozesse innerhalb durchgeführt werden, hierzu werden vier Szenarien vorgeschlagen:

1. Baseline Szenario:
Ein Base-Line-Test wird in der Regel mit erheblicher, aber nicht unüblicher Last durchgeführt. Dieser Test bildet den Maßstab für weitere Tests, um die Leistung der Anwendung zu vergleichen.
2. Last Test Szenario
Lasttests werden normalerweise durchgeführt, um das Verhalten der Anwendung unter einer Spitzenlast zu verstehen. Diese Last kann die gleichzeitige Spitzenanzahl von Benutzern sein, die eine bestimmte Anzahl von Transaktionen innerhalb einer bestimmten Zeitspanne durchführen.
3. Endurance Szenario
Ausführen der Anwendung mit 50 % Last für längere Zeit, um zu prüfen, ob die Anwendung der erwarteten Last standhält.
4. Stresstest Szenario
Ausführen der Anwendung mit mehr als Spitzenlast, um den Punkt zu identifizieren, an dem das System/die Anwendung die Last nicht bewältigen kann.

Die einzelnen Szenarien der Last Tests sind noch hinsichtlich der Anwenderzahlen, der Laufzeit und der von den Anwendern getätigten Prozesse sowie der erwarteten Systemantwortzeiten zu definieren. Es wird empfohlen, mit den LuPs bereits in der SIT Phase zu starten.

3.11.2 Penetrationstest

Ein Penetrationstest (Pentest), ist Teil eines Sicherheitstests mit dem Ziel der Prüfung der Sicherheit möglichst aller Systembestandteile und Anwendungen eines Softwaresystems mit Mitteln und Methoden, die ein Angreifer anwenden würde, um unautorisiert in das System einzudringen (Penetration). Der Pentest ermittelt somit die Empfindlichkeit des zu testenden Systems gegen derartige Angriffe.

Dieser Test muss durch eine externe Fremdfirma unternommen werden. Diese wird nach Anfrage durch das Projekt über den Bereich IT-Sicherheit im ADAC beauftragt.

3.11.3 Failover Tests

Ein Failover Test, respektive Ausfallsicherungstest, umfasst den ungeplanten Wechsel zwischen zwei oder mehreren Netzwerkdiensten oder Instanzen bei einem teilweisen oder vollständigen Ausfall von Komponenten, Diensten, Schnittstellen, Instanzen, etc. Als Folge können die Dienste respektive Instanzen trotz des Ausfalls eines der Systeme hochverfügbar gehalten werden.

INSIS wird grundsätzlich als ein hochverfügbares System betrachtet und muss entsprechend getestet werden.

3.11.4 Business Continuity / Disaster-Recovery (DR) Tests

Disaster Recovery bzw. Notfallwiederherstellung bezeichnet Maßnahmen, die nach einem Ausfall von Komponenten eingeleitet werden und im Rahmen des Betriebstests durchzuführen die im Wiederanlaufplans (Business Continuity Plan) definiert sind. Dazu zählt u.a. sowohl die Datenwiederherstellung,

das „Wiederhochfahren“ von kritischen Applikationen und die Fortsetzung des Betriebs nach der Serviceunterbrechung und ggf. auch das Ersetzen nicht mehr benutzbarer Softwareteile.

Umfang und Zeitpunkt der Planung und Durchführung der DR Tests wird bei der konkreten Release Planung in Absprache mit der Projekt- und Betriebsleitung bestimmt.

3.11.5 Schnittstellenausfalltests

Da die Plattform INSIS an viele Umsysteme angeschlossen sein wird, ist im Rahmen von Schnittstellenausfalltests das Systemverhalten bei Ausfall einzelner Schnittstellenservices zu testen.

Die Entscheidung in welchem Umfang und Rahmen Schnittstellenausfalltests durchzuführen sind, wird im jeweiligen Release Testkonzept oder/ und späteren Betriebskonzept festgelegt.

3.12 Allgemeine Testtechniken

3.12.1 Statische Reviews

Um mögliche Fehler so früh wie möglich zu finden, werden User Stories, Testfall Herleitungen (logische Testfälle) und explizite Testfallbeschreibungen so frühzeitig wie möglich statischen Reviews, Walkthroughs und Inspektionen unterzogen.

3.12.2 Dynamische Black-Box Tests

Um Fehlerwirkungen durch Ausführen eines Testobjekts ausfindig zu machen, werden dynamische Testtechniken genutzt, um Fehlerwirkungen durch Ausführen eines Testobjekts ausfindig zu machen.

3.12.3 Exploratives und strukturiertes Testen

In allen Teststufen können erfahrungsbasierte dynamische Testverfahren zum Einsatz kommen, z.B. das explorative Testen, um die Intuition und die Erfahrung von Experten aus den Fachbereichen oder auch Fachfremden (Monkey Tests) wirksam einzusetzen.

Neben der strukturierten Testfallermittlung können auch Checklisten basierte Tests (auch in Azure DevOps zu erfassen) zum Einsatz kommen, bei dem eine Liste von Kontrollpunkten genutzt wird, wie z.B. Regeln und Kriterien.

4 Testumgebungen

Dieser Abschnitt widmet sich der Planung, Beschreibung und Reservierung der Testumgebung, in der die Tests pro Testphase durchgeführt werden sollen.

4.1 Berechtigungskonzept

In einem Berechtigungskonzept werden Zugriffsregeln für einzelne Benutzer oder Benutzergruppen auf Funktionen und Datensätze des Systems festgelegt. Außerdem werden dort alle Prozesse, die die Umsetzung des Berechtigungskonzepts betreffen beschrieben, wie z.B. das Löschen und Erstellen von Nutzern und Ändern von Berechtigungen. Auch für die Durchführung von Tests sollen Rollen pro Testumgebung definiert werden, an denen Berechtigungsprofile hängen.

Da Tester grundsätzlich auf mehrere Testumgebungen und mit potenziell allen Rollen der künftigen User testen müssen, ist es erforderlich, bei der Konzeption der Systemanmeldung für nicht-produktive Umgebungen einen einfachen und bedienerfreundlichen Rollenwechsel für Tester vorzusehen. Die genaue Festlegung kann in dem Releasetestkonzept erfolgen. Beispiel: Die Authentifizierung für INSIS erfolgt gegen das AD, die Rechtezuteilung innerhalb der Anwendung erfolgt bei der Anmeldung am System.

Ein Testbenutzer hat dabei die Möglichkeit alle notwendigen Rollen einzunehmen, um sicherzustellen, dass die dediziert zugeteilten Rechte und Rollenbündel für die User greifen. Dies dient dem Testen der zukünftigen Autorisierungs- und Authentifizierungsmechanismen sowie der für den Produktivbetrieb angedachten Rollen.

Folgende Punkte müssen u.a. bedacht werden: der Prozess für Neuanlagen (wie wird beantragt, wie genehmigt und wer ist zuständig für das Anlegen der neuen Nutzer), die Regelungen der Rechtevergabe, die Festlegung wie Berechtigungen für das System definiert sind (auf welcher Ebene greifen sie etc.), die Rollenprofile für die Nutzer, ggf. Konzepte für Vertretungsfälle (zeitweise Übertragung der Rechte oder Doppelvergabe) und Regelungen wie das Dokument aktualisiert und kommuniziert wird.

4.2 Testumgebungsmanagement

Testumgebungsmanagement wird beim ADAC dezentral betreut.

Der Umgebungsmanager in Zusammenarbeit mit dem Testmanager und Testkoordinator ist dafür verantwortlich, dass die Testumgebungen rechtzeitig für alle geplanten Testaktivitäten innerhalb einer bestimmten Testphase eingerichtet und reserviert werden. Jede Umgebung muss für die geplanten Aktivitäten geeignet sein.

Sobald die konkreten Testumgebungen bekannt sind, müssen diese an dieser Stelle im Releasetestkonzept benannt werden. Die weitere Detaillierung und Ausarbeitung der notwendigen Prozesse und Artefakte erfolgt im Testumgebungskonzept.

Den Projekten und Plattformen steht es frei, wie sie sich untereinander dazu abstimmen. Es gibt Empfehlungen aus der virtuellen ADAC Organisationseinheit „Querschnitt Test“.

Jede Umgebung muss für die geplanten Aktivitäten geeignet sein, wobei zu beachten ist, dass jede Umgebung nicht nur dem Test Stream und seinen Benutzern zur Verfügung steht, sondern dem gesamten Projekt respektive auch anderen Projekten. Auch hier ist eine gute Kommunikation und eine enge Zusammenarbeit zwischen den anderen Workstreams (insbesondere Infrastruktur und Migration) sowie anderen Projekten erforderlich, um Interessenskonflikte zu vermeiden.

Die weitere Detaillierung und Ausarbeitung der notwendigen Prozesse und Artefakte erfolgt im Testumgebungskonzept.

4.2.1 Übersicht geplante Testumgebungen und Staging

Für das Programm VITA sind folgende Testumgebungen geplant sowie ein möglicher Deployment Plan dargestellt:

ADAC Umgebung	Server	Serverinstanzen	INSIS Testumgebung
DEV	DEV	DEV DEV	DEV PRE-SIT
SIT	TEST	SIT SIT	TEST QA (Time Travelling)
INT	UAT	INT INT INT	INT (Time Travelling Bedarf zu klären) TRAIN PRE PROD
PROD	PROD	PROD	PROD

Tabelle 7: Umgebungsübersicht / technische Systemlandschaft

Die Installation der einzelnen Testumgebungen erfolgt nach Möglichkeit über automatisierte Deployment Prozesse.

In Tabelle 7 werden die Umgebungen so dargestellt, wie sie aus der Perspektive des Test Streams (erste Spalte) und aus der Perspektive des DevOps-Streams (zweite Spalte) aufgerufen werden.

Es ist wichtig zu wissen, dass in jeder Umgebung mehrere Instanzen der Software bereitgestellt werden können. In der DEV-Umgebung laufen zum Beispiel zwei Instanzen auf dem Server, die erste wird vom DevOps-Team für Entwickler- / Unit-Tests verwendet, die zweite vom Test Stream für die InSprint Systemtests. Weitere Details zu diesem Setup werden in Tabelle 7 dargestellt.

Falls zusätzliche Umgebungen oder Instanzen benötigt werden, ist das Infrastruktur-Team zu kontaktieren. Es ist zu beachten, dass jeder zusätzliche Server zu zusätzlichen Kosten führt. Die aktuellen Bereitstellungspläne sehen die vier oben genannten Umgebungen mit dem folgenden Verfügbarkeitschema vor (Stand April 2021):

DEV	- Verfügbar ab Anfang Juni 2021
SIT	- Lieferbar ab Juli/August 2021
INT	- Lieferbar ab Anfang Dezember 2022
PROD	- Verfügbar für Pre GoLive-Testing kurz vor dem ersten Go-Live des MVP (März 2022)

Falls zusätzliche Umgebungen oder Instanzen benötigt werden, ist das Infrastruktur-Team zu kontaktieren. Es ist zu beachten, dass jeder zusätzliche Server zu zusätzlichen Kosten führt.

4.2.2 Staging Path und Deployment-Strategie

Der Staging-Pfad ist wie folgt definiert: DEV > TEST > SIT > UAT > PreProd > PROD

ADAC Umgebung	Teststufe	Beschreibung	Deployment
DEV	Unit	Entwicklungsumgebung	Entwicklerentscheidung
PRE-SIT	InSprint	InSprint Test Umgebung	Nächtlich / Anforderung
TEST	Systemtest	System übergreifend mit Mocks	2 - 3 pro Woche in Abstimmung
QA	Systemintegrationstest	Teilw. Anbindung Umsysteme	1 - 2 pro Sprint / wöchentlich
INT	SIT /UAT	User Acceptance Tests, E2E	nach Bedarf
TRAIN	-	Trainingsumgebung	nach Bedarf
PRE PROD	-	L&P, PEN, (SEC)	nach Bedarf
PROD	Post Go-live	Post-installation Smoketests	Per Release

Tabelle 8: Übersicht Anwendungszwecke der unterschiedlichen Instanzen

Damit die UAT- und Performance-Tests zu einer repräsentativen Benutzererfahrung bzw. zu repräsentativen Performance-Angaben führen, müssen die INT- und PRE PROD-Testumgebungen identisch oder möglichst identisch mit der späteren Produktionsumgebung spezifiziert sein.

Die Installation der einzelnen Umgebungen erfolgt über automatisierte Deployment Prozesse. Automatisierte De-Implementierungen werden über das Azure DevOps GIT durchgeführt.

4.2.3 Testziele / Testdurchführung pro Umgebung

Umgebung	DEV			SIT			UAT				PROD		
Testumgebung	DEV	PRE-SIT		TEST	QA		INT	TRAIN	PreProd		PROD	Anmerkung	
Functional Test													
InSprint-Test	X	X											
SIT		X		X	X								
UAT					X		X						
PreGolive									X		X		
Automated Test													
Smoke Tests		X		X	X		X	X	X		X		
Regression Tests		X		X	X		X	X	X		X		
NFT													
LUP				X	X		X		X			Notwendigkeit für Test auf isolierten Umgebungen (TEST und QA) zu prüfen	
PEN							X		X		X	Durch IT-Sicherheit zu entscheiden	
Failover & Component Failure				X	X		X		X			Notwendigkeit für Test auf isolierten Umgebungen (TEST und QA) zu prüfen	
DR							X		X				
Additional Testing												Zu entscheiden	

Tabelle 9: Testziele pro Umgebung

Die obige Tabelle stellt eine Übersicht über die möglichen Testaktivitäten / Teststufen pro Testumgebung dar. Im Rahmen des Testplans wird definiert, welche Testumgebungen im Einzelnen zu verwenden sind.

4.3 Qualitätsmerkmale

Hier wird beschrieben, welche Qualitätskriterien von welcher Instanz und mit welcher Priorität im Test nachgewiesen werden. Grundlage für die Beschreibung sind die Kriterien aus dem Qualitätsmodell der Norm ISO 9126.

Die ADAC Test-Policy ([hier](#)) listet alle Qualitätsattribute auf, die für die Testprozesse bei ADAC-Projekten relevant sind. Die [Tabelle 9: Testziele pro Umgebung](#) bietet eine konkretere Beschreibung der Relevanz der einzelnen Attribute im Rahmen des Testablaufs für das Projekt VITA.

Nicht alle Qualitätskriterien werden oder können getestet werden bzw. sind im Testumfang enthalten. Nachfolgend werden die Qualitätskriterien genannt, welche im Rahmen von VITA aus den unten explizit genannten Gründen reduziert oder nicht relevant sind.

4.3.1 Abgrenzung: Qualitätsmerkmale, die nicht getestet werden

Nicht alle Qualitätskriterien werden oder können getestet werden bzw. sind im Testumfang enthalten. Nachfolgend werden die Qualitätskriterien genannt, die im Rahmen von VITA aus den unten explizit genannten Gründen reduziert oder nicht relevant sind.

Funktionalität / Richtigkeit des Standardproduktes

Die im Rahmen des Projektes modifizierten Funktionalitäten werden, wie in diesem Konzept beschrieben nach einem Anforderungs- und risikobasierten Prinzip getestet, um die korrekte Implementierung durch das SCRUM Team zu verifizieren. Je nach Kapazität und Risiko wird sich die Testabdeckung jedoch auf wesentlichen Szenarien beschränken.

Da es sich um ein Standardprodukt handelt, werden zusätzliche Tests der INSIS-Kernfunktionalität vom Projektteam nicht benötigt und die Ergebnisse der implementierten Konfigurationen konsistent und korrekt sind.

Usability / Benutzbarkeit

Beinhaltet die Attraktivität, Bedienbarkeit, Erlernbarkeit und Verständlichkeit von INSIS.

Da es sich bei dem System um ein Standardprodukt handelt, das zu einem sehr hohen Maße lediglich konfiguriert werden soll, ist der Einfluss auf die oben genannten Faktoren gering.

Das Testen der vom System bereitgestellten Standardfunktionalität liegt daher außerhalb des Rahmens.

Ergeben sich im Verlauf der geplanten Testaktivitäten, wie z.B. im Rahmen der Prozesstests, Einblicke und Hinweise zur Benutzerfreundlichkeit, so werden diese an entsprechenden Workstreams und Teams zur Nutzung in Benutzerschulungen und Benutzerdokumentation weitergeleitet.

Übertragbarkeit & Austauschbarkeit

Der Umfang des Projekts ist die Implementierung von INSIS, nicht dessen Übertragbarkeit oder Austauschbarkeit und daher sind diese Aspekte nicht im Scope der Testaktivitäten.

Installierbarkeit

Probleme, die während des Installationsprozesses und der Staging-Aktivitäten gefunden werden, werden für zukünftige Referenzen dokumentiert, aber der Prozess wird nicht aktiv getestet. Die Software-Suite als Ganzes wird so geliefert, wie sie ist, was auch für ihre Installierbarkeit gilt.

Sonstige Qualitätsmerkmale

Änderbarkeit, Wartbarkeit, Prüfbarkeit, Testbarkeit, Analysierbarkeit und Modifizierbarkeit sind als Testkriterien nicht relevant, da die Software als Standardpaket ausgeliefert wird, wobei ein ausreichendes Qualitätsniveau auch in diesen Punkten durch den Lieferanten garantiert wird.

4.4 Kriterien: Eingang, Ende, Abnahme, Abbruch, Wiederaufnahme

4.4.1 Eingangskriterien

Hier wird mittels geeigneter Eingangskriterien beschrieben, wann mit dem Test der jeweiligen Stufe begonnen werden kann. Auch die Form der Prüfung und die durchzuführenden Maßnahmen bei Nichterfüllung eines Eingangskriteriums werden für die einzelnen Testinstanzen und -arten dokumentiert.

Über alle Teststufen hinweg gelten folgende Eingangskriterien:

- Testfälle und Testszenarien sind entwickelt
- zu testende Software(-teil) steht zur Verfügung
- zu testende Schnittstelle ist an Um Systeme angebunden bzw. Mockups sind installiert
- User mit zugehörigen Rollen und Rechten für die zu testende Software definiert
- Testumgebung steht bereit

- Testdaten stehen zur Verfügung
- Bereitschaft der Testwerkzeuge & Ressourcenverfügbarkeit ist gegeben
- Anbindung an externe Anbieter/Cloud ist vorhanden

Außerdem wird davon ausgegangen, dass das Projekt eine (ausreichend) vollständige und korrekte Testbasis bereitstellt.

Die Eingangskriterien variieren je nach Teststufe.

Teststufe	Testeingangskriterium	Prüfung / Grenze	Maßnahmen bei Nichterfüllung
In Sprint			
Deployment	Alle Systemmodule sind für das Testen verfügbar	Die Umgebung und das System lassen sich aufrufen	• Eintrag im Fehlermanagement • Fehlerbehebung durch Entwickler • Erneute Systemlieferung
Umgebungsbereitschaft	Das Deployment in die Testumgebung wurde abgeschlossen.	Prüfung durch Testumgebungsmanager	• Eintrag im Fehlermanagement • Fehlerbehebung durch Entwickler
	Die Testdaten und -umgebung stehen bereit. Das Scrum Team des jeweiligen Work Streams hat den Anforderungen entsprechende Testfälle beschrieben.	Prüfung durch Testumgebungsmanager Der Scrum Tester (Teil des Scrumteams) prüft den Testfallstatus in Azure DevOps-	• Entscheidung, ob und mit welchen Testfällen weiter getestet werden soll
Teststufe	Testeingangs-Kriterium	Prüfung / Grenze	Maßnahmen bei Nichterfüllung
SIT / Prozesstest			
Deployment	Alle relevanten Systemmodule sind für das Testen verfügbar	Die Umgebung und das System lassen sich aufrufen	• Eintrag im Fehlermanagement • Fehlerbehebung durch Entwickler • Erneute Systemlieferung
Lieferungen aus der vorherigen Teststufe	Relevante InSprint-Testfälle, die die zu testenden Schnittstellen betreffen	100%	• Eintrag im Fehlermanagement
	Offene 'Critical' Defects, die derzeit überprüft/behoben werden	< =5%	• Bericht an Testmanager • Ggf. Eskalation an PL
	Offene 'High' Defects, die derzeit überprüft/behoben werden	< = 10%	
	Die vorherige Teststufe ist abgeschlossen	Abgeschlossen durch Testmanager	
Testplanung und Vorbereitung	Testfalldesign und Testplan	Abgeschlossen durch Testmanager	
Umgebungsbereitschaft	Verfügbarkeit, Stabilität und Leistung der Testumgebung Testumgebung wie im Testumgebungskonzept festgelegt	Abgestimmt Muss verfügbar sein	• Eintrag im Fehlermanagement • Bericht an Testmanager und Testumgebungsmanager
	Testdaten in der erforderlichen Qualität und Quantität	Muss verfügbar sein	• ggf. / Eskalation an PL
Teststufe	Testeingangs-Kriterium	Prüfung / Grenze	Maßnahmen bei Nichterfüllung
UAT			
Deployment	Alle Systemmodule sind für das Testen verfügbar	Die Umgebung und das System lassen sich aufrufen	• Eintrag im Fehlermanagement • Fehlerbehebung durch Entwickler • Erneute Systemlieferung

Teststufe	Testeingangskriterium	Prüfung / Grenze	Maßnahmen bei Nichterfüllung
Lieferungen aus der vorherigen Teststufe	Verfügbarkeit der Systemintegration	100%	Eintrag im Fehlermanagement
	System-Integrationstestfälle bestanden	> 95%	Bericht an Testmanager
	Prozesstests bestanden	> 95%	Ggf. Eskalation an PL
	Offene 'Critical' Defects, die derzeit überprüft/behoben werden	< = 1%	
	Offene ‚High‘ Defects, die derzeit überprüft/behoben werden	< = 5%	
Testplanung und Vorbereitung	Testfalldesign und Testplan	Abgeschlossen durch Testmanager	- Bericht an Testmanager - Ggf. Eskalation an PL
	Testfalldesign und Testplan	Abgestimmt	
Umgebungsbereitschaft	Verfügbarkeit, Stabilität und Leistung der Testumgebung sowie Zeitaufwand für die Fehlerbehebung	Abgestimmt	- Eintrag im Fehlermanagement - Bericht an Testmanager und Testumgebungsmanager
	Testumgebung wie im Testumgebungskonzept festgelegt	Muss verfügbar sein	ggf. / Eskalation an PL
	Testdaten in der erforderlichen Qualität und Quantität	Muss verfügbar sein	
	Anwendungsbenutze) mit den gewünschten Rollen und Berechtigung	Muss verfügbar sein	
GoLive			
Deployment	Alle Systemmodule sind für das Testen verfügbar	Die Umgebung und das System lassen sich aufrufen	- Eintrag im Fehlermanagement - Bericht an Betrieb und Projektleitung
Go live Tests	Go live Tests nach Deployments erfolgreich	Die Umgebung und das System lassen sich aufrufen	- Eintrag im Fehlermanagement - Bericht an Betrieb und Projektleitung - Fehlerbehebung

Tabelle 10: Eingangskriterien

* Prozentzahlen für Defects werden im jeweiligen Release Testkonzept nachgeschärft, die vorliegenden Zahlen dienen als Orientierungsrahmen.

4.4.2 Testende- / Abnahmekriterien

Die Testendekriterien variieren je nach Teststufe.

Teststufe	Testausgangskriterium	Prüfung / Grenze	Maßnahmen bei Nichterfüllung
InSprint			
Testausführung	Testausführung	100%	
	Testfälle bestanden	> 80%	• Die Anzahl an nicht abgenommenen US/Sprint muss kontrollierbar bleiben
	Offene 'Critical' Defects, die derzeit überprüft/behoben werden	<= 5%	• Bericht an Testmanager • Ggf. Eskalation an PL
	Offene 'High' Defects, die derzeit überprüft/behoben werden	<= 10%	• Ggf. Eskalation an PL
SIT			

Teststufe	Testausgangskriterium	Prüfung / Grenze	Maßnahmen bei Nichterfüllung
Testausführung	Testausführung	100%	• Eintrag im Fehlermanagement
	Testfälle bestanden	> 85%	• Bericht an Testmanager
	Offene 'Critical' Defects, die derzeit überprüft/behoben werden	<= 2%	• Ggf. Eskalation an PL
	Offene 'High'-Fehler, die derzeit überprüft/behoben werden	<= 5%	
UAT			
Testausführung	Testausführung	100%	• Eintrag im Fehlermanagement
	Testfälle bestanden Prozent	> 95%	
	Offene 'Critical' Defects, die derzeit überprüft/behoben werden	<= 1%	
	Offene 'High'-Fehler, die derzeit überprüft/behoben werden	<= 1%	
	Alle ungelösten Fehler sind vollständig dokumentiert oder in bekannte Fehler überführt und es wurde vereinbart, dass sie in späteren Releases als Änderungsanforderung gelöst werden.	Erledigt	
Sign-off	Die voll integrierte Lösung wurde vom Testmanager als produktionsreif empfohlen.	Erledigt	• Bericht an Testmanager • Ggf. Eskalation an PL
GoLive			
Go live Tests	Blockieren Fehler (Showstopper) BZP	0%	• Beurteilung / Risikobewertung der Fehler im Fehlergremium • Eskalation an PL • Ggf. Kommunikation an Benutzerkreis

Tabelle 11 Testende / Abnahme Matrix

4.4.3 Abbruch und Wiederaufnahmekriterien

Treten Prio 1 Fehler auf, die Test- oder Go live verhindernde Showstopper sind, sind diese sofort mit dem Testkoordinator und Testmanager zu besprechen, um festzustellen, welche Aspekte noch getestet werden können, um ein aussagekräftiges Feedback über die Qualität der gelieferten Funktion für andere Aspekte zu erhalten.

Sobald der Showstopper als behoben gemeldet wird, wird das Testen wieder aufgenommen.

Um zu prüfen, dass der Fehler tatsächlich behoben wurde, werden die Tests erneut durchgeführt, bei denen der Fehler auftritt, sowie die zugehörigen (abhängigen) Testfälle. Um Regressionseffekte ausschließen zu können, kann es sinnvoll sein, eine geeignete Auswahl (z.B. nur Prio 1 Regressionstests des betroffenen Bereichs) erneut auszuführen.

Fehler mit hoher Kritikalität verhindern, dass ein signifikanter Teil einer Funktion wie erwartet funktioniert, und / oder verhindern, dass sie getestet werden können. Zumeist ist eine Weiterarbeit ggf. mit einem Workaround möglich.

Das Testen dieses betroffenen Teils der Funktionalität wird gestoppt. Hier entscheidet der Testmanager über die weitere Fortführung des Tests. Hält er es für lohnend, zusätzlichen Testaufwand zu investieren, um potenzielle zusätzliche Probleme oder zusätzliche Details zu dem gefundenen Problem aufzudecken, wird die Testdurchführung mit dem nächstmöglichen Testschritt fortgeführt.

Nicht betroffene Aspekte können weiterhin getestet werden, jedoch sind alle betroffenen Testfälle mit hoher Priorität nach Installation einer neuen Version bzw. der Fehlerkorrektur noch einmal auszuführen, um sicherzustellen, dass in diesen Aspekten keine Regression aufgetreten ist.

Fehler mit mittlerer und niedriger Kritikalität rechtfertigen keinen Stopp der Testaktivitäten, es sei denn, ihre Anzahl übersteigt eine kritische Zahl. Wenn der Umfang der erneut durchzuführenden

Wiederholungstest (Defect Retests, erneute Testfalldurchführung und ggf. Regressionstests) den Nutzen weiterer Tests (z.B. weiteres Finden von Defects) überwiegt, entscheiden der Testmanager, die konkret betroffenen Testaktivitäten zu stoppen ist, bis ein ausreichendes Qualitätsniveau erreicht ist.

5 Testanalyse und -design

5.1 Testbasis

Bevor mit den Testaktivitäten begonnen werden kann, muss die Testbasis vorhanden sein. Die Testbasis unterscheidet sich je nach Art des Testobjekts (z.B. Akzeptanzkriterien in den US, Konzepte, Prozessbeschreibungen, Schnittstellenbeschreibungen, etc.).

Die Informationen zu der Testbasis wie z.B. Anforderungen Epics, Features, User Storys und Integrationspunkte werden im Azure DevOps und Sharepoint / MS Teams abgelegt:

- INSIS – Azure DevOps Service (Bereitstellung Anfang Juni 2021)
- Umsysteme (wie zb. BSI, FS-CD, Alina Wanda, BI, adac.de, Raps, etc. (siehe [hier](#)) – Azure DevOps Server

Es wurde entschieden, die systemübergreifenden Tests für die INSIS / FS-CD Integration in Azure DevOps abzulegen.

5.2 Qualitätssicherung der Testbasis

Es wird vorausgesetzt, dass User Stories und Akzeptanzkriterien vollständig und korrekt sind und vom Product Owner der jeweiligen Work Streams bestätigt werden.

Ein Review der Testbasis wird vom Testteam durchgeführt, das nach den Erfordernissen der jeweiligen Teststufe zusammengesetzt ist (rund um ein fixes Kernteam inkl. Scrum Testern), um mögliche Lücken zu identifizieren und fehlende Informationen, Inkonsistenzen oder Unstimmigkeiten aufzudecken und deren Korrektur oder Ergänzung einleiten zu können. Die Testanalyse stellt auch eine statistische Qualitätssicherung dar, die vorzugsweise vor Beginn der Entwicklung und ggf. auch während der Entwicklung durchgeführt wird.

5.3 Schnittstellenintegration

Um ein effektives Testdesign für Schnittstellenverbindungen durchführen zu können, werden folgende Artefakte als Teil der Testbasis benötigt:

- Datenwörterbuch (Beschreibung der zu integrierenden Umsysteme)
- Schnittstellen-Spezifikation

In der Schnittstellen-Spezifikation (traditionelles Liefermodell) bzw. in dem Integrationspunkt wird beschrieben, wie die Datenübertragung erfolgt und wie die von einem System übertragenen Daten auf die Attribute einer angeschlossenen Schnittstelle abgebildet werden sollen. Dieses Artefakt beschreibt das Mapping einschl. Transformationsregeln wie z.B. möglicher Verkettungen, Verkürzungen oder anderer Formen der Umwandlung auf Basis der jeweiligen Systemanforderungen.

Falls es Priorität und Kritikalität des Tests erfordern, kann eine detaillierte Grenzwertanalyse auf Basis der Schnittstellenbeschreibung zur systematischen Testableitung verwendet werden.

5.4 Funktionale Workflows

Funktionale und technische Workflows werden innerhalb der INSIS-Software-Suite konfiguriert, um die funktionalen Anforderungen der priorisierten User Stories abzudecken.

Ein Review der vollständigen Testbasis wird vom Testteam durchgeführt, um mögliche Lücken zu identifizieren und fehlende Informationen, Inkonsistenzen oder Unstimmigkeiten aufdecken und deren Behebung einleiten zu können. Diese Anforderungen bilden also nicht nur den Input für das Testdesign, sondern bilden ebenfalls die Basis für den Testworkflow. Die Analyse im Vorfeld bildet auch eine wichtige Phase des statischen Testens, während und vorzugsweise vor Beginn der Entwicklung.

5.4.1 Test Workflow in InSprint Stage

Die Aktivität der Testvorbereitung während des InSprint Tests wird von den SCRUM Testern für jedes SCRUM Team durchgeführt und beinhaltet die Erstellung von Testszenarien, Testdatenanforderungen und Testfällen.

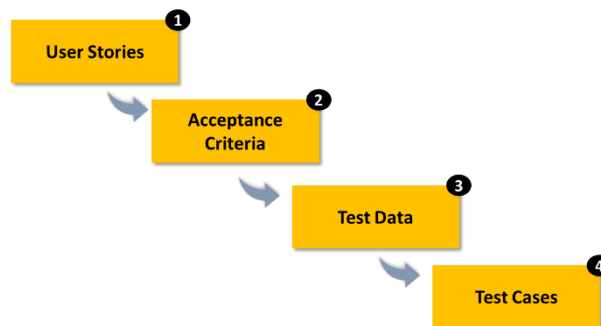


Abbildung 5: Testworkflow in InSprint Stage

Die Testfälle werden für das InSprint Testing anhand der Akzeptanzkriterien der einzelnen User Stories definiert. Hierzu sind auch „Platzhalter“ Testfälle zulässig, die lediglich die Akzeptanzkriterien abtesten. Im Fehlerfall wird aus dem Testfall ein Defect erstellt. Die zugehörige User Story wird nicht geschlossen, sondern wieder dem Entwickler zugewiesen.

Um eine User Story zu schließen, sind die Kriterien zu erfüllen, welche in der Definition of Done (DoD) (wie z.B. Schließen von bestimmten Tasks) beschrieben werden. Alle einer Userstory zugeordneten Task müssen dabei geschlossen („done“) sein um die US selbst abnehmen zu können.

Die nachfolgende Übersicht zeigt den Übergang und die Statuswerte einer User Story.

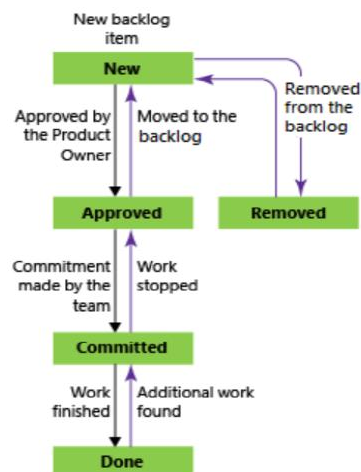


Abbildung 6: Userstory

Status	Assigned To	Bearbeiter Bedeutung
New	Product Owner, Scrum Master oder Business Analyst	Eine Anforderung ist erfasst
Approved	PO	Die Anforderung ist fachlich abgestimmt
Committed	Entwickler	Die Entwicklung plant die Umsetzung ein. Dieser Status ist im Zuge des Deployments mit den Informationen zur Build-Bezeichnung zu ergänzen.
In Test	(Von Entwickler an) Tester	Bereit zum Testen / Test begonnen
Completed	PO	Der Testfall/ Testfälle konnten durchgeführt und waren erfolgreich
Done	PO	Nur nach einem erfolgreichen Test kann die Anforderung abgenommen/ geschlossen werden
Removed	Product Owner	Die Anforderung wird nicht mehr benötigt

Tabelle 12 : Ablauf Implementierung User Story

Falls für die Arbeitssteuerung der Implementierung mit der Anlage von Tasks gearbeitet wird, kann beispielsweise festgelegt werden, dass ein Tester mit dem Schließen des designeden Testtasks den erfolgreichen Test der entsprechenden User Story bestätigt. Alternativ können feiner abgestufte Statuswerte wie z.B. „In Development“ und „In Test“ verwendet werden.

5.4.2 Test Workflow ab SIT

Der typische Testprozess in der SIT- und UAT-Phase ist ähnlich wie in der InSprint-Phase mit der Ergänzung um Quality Gates .

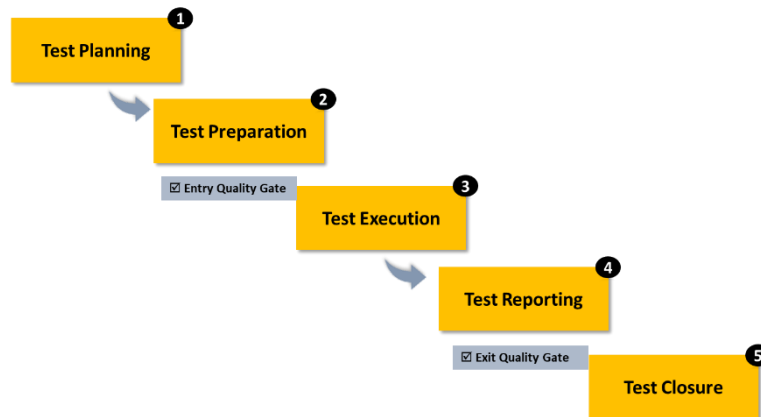


Abbildung 7: Test Workflow ab SIT

Die Zielsetzung einer systematischen Testfallermittlung für Prozess- und Schnittstellen-tests in SIT und UAT ist es, eine erhöhte Abdeckung der fachlichen Anforderungen zu erreichen, um bei der Testausführung die kritischen Fehler möglichst frühzeitig zu finden. Dabei ist darauf zu achten, dass zuvorderst eine Abdeckung von geschäftskritischen Prozessen oder Teilprozessen („Rundlauftests“ bzw. e2e Tests) erfolgt, d.h. es wird der Ansatz des risikobasierten Testens gewählt. Dies dient auch dem Erkennen von fehlerhaften, unvollständigen oder ungenügenden Anforderungen.

Bei der Testfallerstellung wird vor allem auf die „anwendungsfallbasierte Testfallermittlung“ gesetzt.

Die impliziten Anforderungen an das Programm werden dabei in kritischen Bereichen durch die Testfälle mit getestet. Eine Ergänzung aus zufallsbasierter und intuitiver Testfallermittlung wird durch exploratives Testen erreicht.

Wird ein Testfall nicht erfolgreich durchgeführt, wird aus dem Testfall ein Defect erstellt.

5.5 Testfallermittlung und Testtiefe

Die Teststrategie für das Projekt ist sowohl risiko- als auch anforderungsbasiert. Für die Testfallerstellung bedeutet dies, dass im Sinne eines anforderungsbasierten Tests, Testfälle aus den Akzeptanzkriterien der in User Stories oder Integration Points (Schnittstellen) beschriebenen Anforderungen abgeleitet werden.

Im Sinne des risikobasierten Ansatzes werden die Testfälle anhand Ihrer Kritikalität priorisiert, so dass kritische Programmfunktionen möglichst früh und intensiv getestet werden. Wenn aus zeitlichen Gründen nicht alle Testfälle durchgeführt werden können, sollten dies gem. dem risikobasierten und agilen Ansatz nur Testfälle mit niedriger Priorität und für nicht geschäftskritische Funktionen sein.

Für die entsprechend festgelegten Kritikalitäts-Stufen soll zudem beschrieben werden, welche Methoden für die Testfallermittlung Verwendung finden und welche Testtiefe zu erreichen ist.

Zielsetzung einer systematischen Testfallermittlung ist es, eine möglichst hohe Testabdeckung der fachlichen Anforderungen zu erreichen. Grundsätzlich wird der „vorbeugende Ansatz“ angewendet. Das heißt, Testaktivitäten wie Reviews und Testfallerstellung starten so früh wie möglich. Im konkreten Fall bedeutet das, dass die Testfallerstellung im Zuge der Formulierung von Anforderungen erfolgt (abgeleitet aus Akzeptanzkriterien als Dummy Testfall oder konkreter Testfall mit einzelnen Testschritten). Durch dieses Vorgehen können fehlerhafte oder ungenügende Anforderungen erkannt und verbessert werden.

Die systematische Testfallermittlung wird um eine zufallsbasierte, intuitive Testfallermittlung in explorativen Test ergänzt.

Testfälle für Prozesstests werden mittels einer Szenario Planung ermittelt, die das Ziel verfolgt, die Prozessschritte über Schnittstellen hinweg durch die Verkettung von Funktionalitäten –in repräsentativen fachlichen Ausprägungen – zu überprüfen.

5.6 Testobjekte

5.6.1 Testumfang

Der Umfang des Testens wird auf Basis der Beschreibung des Testobjekts und den Akzeptanzkriterien DoD (Definition of Done) bestimmt, welche die funktionalen und technischen Anforderungen abbilden.

Die Standardfunktionalität (OOTB-Funktionalität) von INSIS ist im Rahmen des VITA-Projekts nur impliziter Gegenstand von Tests. Sie wird im Rahmen von Prozesstests ebenfalls mitgetestet.

Sollte ein INSIS Core-Fehler gefunden werden, wird dieser an den Lieferanten zur Klärung übergeben. Solche Defects bilden eine eigene Kategorie in der Azure DevOps Fehlerübersicht.

5.7 Erstellung der Testfälle

Nach Planung und Entwurf des Tests sollten alle Entscheidungen getroffen und Informationen vorliegen, die Voraussetzung für das Erstellen der Testfälle sind. Die Testfälle beschreiben die eigentlichen Tests, d.h. die konkreten Schritte, die ein Tester beim Testen der Anwendung ausführt.

Ausgehend von der Aufteilung der Testdurchführung in mehrere Teststufen werden die Testfälle und Szenarien für die einzelnen Stufen festgelegt.

Die Anzahl der Testfälle und somit der Testaufwand ist in Abhängigkeit zur Teststrategie (Risikobewertung und Budget) zu definieren.

Die Testfälle werden nach Priorität kategorisiert, basierend auf den folgenden Regeln wie in 3.2.2 Testfall Priorisierung beschrieben.

Alle Testfälle, die in Azure DevOps erfasst und gespeichert werden, müssen folgende Informationen enthalten:

- Für InSprint, SIT und LuP: Testfallerstellung in auf Basis der 2-sprachigen Akzeptanzkriterien (Deutsch & Englisch)
- Andere Teststufen: Die Sprache für Testfälle ist auf Basis der Haupttester festzulegen (im Zweifel Deutsch)
- Aspekte zum Testfalldesign
 - Zuordnung zu Anforderungen (Referenz auf PBIs)
 - Zuordnung zu einem Testplan / Testsuite
 - Zu testende Funktionen / Prozesse
 - Testschritte
 - Erwartete Ergebnisse
 - Abhängigkeit zu anderen Testfällen
 - Eindeutige Testfall-ID (wird automatisch erzeugt)
 - Priorität

Wenn ein Fehler z.B. während der Testausführung auftritt

- Referenzen zu Fehlern

5.8 Vorbereitung der Tests

In den nachfolgenden Projektphasen wird pro Teststufe ein Prozess erarbeitet, wie die Testaktivitäten remote mit den Testaktivitäten vor Ort synchronisiert werden. Es soll definiert werden, bei wem die Verantwortlichkeiten für einzelne Testaktivitäten und Arbeitsergebnisse liegen wie auch die Abläufe und zeitlichen Planungen. Grundsätzlich sind für diese Abstimmungen der ADAC Testmanager sowie der

Testkoordinator verantwortlich. Diese Arbeitsaufteilung wird im Releasetest Konzept näher spezifiziert werden. (siehe auch Kapitel 3.5.2 Verantwortlichkeit von ADAC und Partner)

5.9 Auswahl der Testtools

Der ADAC nutzt Microsoft Azure DevOps als Verwaltungssystem, um Anforderungen, Testartefakte, Fehler/Defects und auch Entwicklungsrepositories zu verwalten. Wie auch in den allgemeinen ADAC-Testrichtlinien beschrieben, kann es erforderlich sein, dass zusätzliche Plugins installiert werden müssen, um weitere Funktionen für die Bedürfnisse des Projekts bereit zu stellen. Dies umfasst beispielsweise die Aufzeichnungssoftware zur Dokumentation von explorativen Tests.

Hinweis: Gemäß dem Azure DevOps high-level-Konzept ([hier](#)) ist als Betriebskonzept die Nutzung eines Azure DevOps Tenant als „Fully Managed Service“ gewählt (Stand Ende Analysephase).

5.9.1 Testen mit Azure DevOps

Die zentralen Anforderungen an Azure DevOps sind:

- durchgängige Nachverfolgbarkeit von der Anforderung zum Testfall und zurück
- durchgängige Nachverfolgbarkeit von der Anforderung zum Defect und Retest Ergebnis
- die Dokumentation aller Testaktivitäten insbesondere deren Ergebnisse möglichst vollumfänglich und nachvollziehbar im Tool erfolgen
- Auffindbarkeit, Verständlichkeit und Nachvollziehbarkeit der Testdokumentation
- Revisionssicherheit der Dokumentation

Testfälle bilden eine eigene Entität in Azure DevOps und können direkt mit PBIs und Defects in einer 1:0...n-Beziehung verknüpft werden. Die systematische Verwaltung von Testfällen, Defects und Ausführungsergebnissen ermöglichen Abfragen, Dashboards und Berichte, um Ergebnisse und Übersichten zu generieren, die für den Einblick in den Fortschritt und die Projektsteuerung benötigt werden.

Testfälle existieren weder in einem Vakuum noch werden sie automatisch generiert, Azure DevOps ist lediglich ein Repository für diese.

Beim explorativen Testen ist es nicht möglich, jeden Testfall im Voraus vollständig zu spezifizieren und zu dokumentieren. Hier wird sich das Testteam auf Dokumentationsstandards einigen, die mit der Standardfunktionalität von Azure DevOps kompatibel sind. Es können zusätzliche offizielle Plugins zum Einsatz kommen, die explorative Testprozesse unterstützen, indem sie Aktivitäten und Ergebnisse aufzeichnen, Kommentare und Befunde notieren können und es so erlauben, Testergebnisse und Fehler automatisch mit dem Backlog-Element oder der zu testenden Funktionalität zu verknüpfen.

Von allen Testern im Projekt wird erwartet, dass sie ein grundlegendes Verständnis dafür haben, wie Testprozesse innerhalb von Azure DevOps implementiert und befolgt werden. Zudem wird vorausgesetzt, dass der Testmanager und Testkoordinator das Team in die Lage versetzen können, ihre Aktivitäten strukturiert und gut dokumentiert durchzuführen. Gegebenenfalls müssen entsprechende Schulungen durchgeführt werden.

5.9.2 Defect Management mit Azure DevOps

Für das Defect Management wird Azure DevOps das einzige Tool (Single Source of Truth) sein. Azure DevOps bietet die Möglichkeit, Defects zu dokumentieren und nach Risiko, Auswirkung und Priorität zu klassifizieren. Diese Metriken können auch vom Testmanager für das Reporting genutzt werden.

Da alle Anforderungen als PBIs in Azure DevOps erfasst werden und diese in ihrem LifeCycle nachgehalten werden sollen, müssen Defects zu dem betreffenden Testfall respektive PBI, verlinkt werden (siehe Kapitel 3.2 Fehlermanagementprozess).

5.9.3 Product Backlog-Item Management mit Azure DevOps

Projekt-Backlog-Elemente (PBIs) wie Epics, Features, User Stories und Integration Points beschreiben, welche Anforderungen an das Produkt gestellt werden.

Es ist ein wichtiges Prinzip, ein gut strukturiertes, gemeinsames Testfallrepository zu haben, auf das sich alle Projektmitglieder beziehen können. Es ist die Aufgabe jedes Teammitglieds respektive aller Projektmitarbeiter, die Integrität dieses Repositories zu wahren.

Von Beginn des Projekts an und während der gesamten Release-Zyklen liegt es in der Verantwortung des Testmanagers sicherzustellen, dass die Testfälle in Azure DevOps in einer geeigneten Struktur (z.B. funktional) abgebildet ist.

Solche eine hierarchische Struktur repräsentiert das System als Ganzes einschließlich der Anbindung an alle externen Schnittstellen. Für die Strukturierung der InSprint-Tests, die sich an der Userstory Entwicklung orientieren, kann sich auch eine Gliederung nach Sprints bewähren.

Es liegt in der Verantwortung der Business Analysten, sicherzustellen, dass alle Funktionalitäten und Akzeptanzkriterien korrekt und vollständig dargestellt werden, so dass die Tester die User Stories gegen die Testfälle abtesten können, sowie ab der SIT- Phase und bei Prozesstests die Testfälle der richtigen Funktionalität zuordnen können. Auf diese Weise können die Testmanager und Business Analysten leicht auswählen und priorisieren, welche Testfälle bei der Implementierung oder Anpassung eines Features für einen Regressionstests von Relevanz sind. Außerdem können Testmanager durch den Einsatz von Dashboards und Abfragen leichter erkennen, bei welchen Aspekten der zu testenden Funktionalität bzw. in welchem System es zu Verzögerungen oder Problemen aufgrund von Fehlern kommt.

5.9.4 Informationsfluss während der Testaktivitäten

Sobald im weiteren Projektverlauf feststeht, welche Teile des Projektes, welche Teams und welche Personen sich zu Testteams zusammenfinden, wird im Projektkommunikationsplan und Release-Testplan aufgelistet, wie die Informationsflüsse an alle Beteiligten gestaltet werden.

5.10 Testdaten

Für die erfolgreiche Testdurchführung ist es notwendig, geeignete Testdaten zur Verfügung zu stellen. Eine Auspezifizierung findet sich [hier](#).

Seitens des Fachbereichs gibt es folgende Anforderungen an das Testdatenmanagement.

Die konkreten Lösungsansätze sind in den Releasephasen zu erarbeiten und ggf. mit der Projekt- und Programmleitung abzustimmen, da der Scope weit in die ADAC-Welt hineinreicht und damit über das INSIS-Umsetzungsprojekt hinausgeht:

- Mitgliedschaftsdaten, Versicherungsnehmerdaten, Schadendaten kopieren und / oder synthetisch generieren durch den Fachbereich
- Möglichkeit Mock-Daten systemintern zu verwenden und durch FB pflegen zu können
- Zukünftig inklusive Sperrkennzeichen
- GUI / Grafisches Abfragetool für Selektion mit anschließender Kopier- / Exportiermöglich inkl. Anonymisierung, wenn notwendig
- Adaption / Veränderung von bestehenden Daten
- Durchgängige Zeitreisefähigkeit über alle Systeme für End to End Tests

Diese Anforderungen gelten, sobald mit der Implementierung des MVP begonnen wird und für die anschließende künftige Weiterentwicklung von / in INSIS. Weitere Testdatenanforderungen ergeben nach der Analyse der Testfallerfordernisse.

Die Erstellung und Nutzung von Testdaten ist abhängig von der Projektphase und Teststufe. Die Entscheidung, ob die Testdaten erzeugt bzw. erstellt werden oder anonymisierte, produktive Daten zu Testzwecken (z.B. über InPro Antrag) genutzt werden, liegt in der Verantwortung des Testmanagers, der sich mit den Fachbereichen und Product Ownern abstimmt. Für die Freigabe der Nutzung von produktiven Daten zu Testzwecken ist die Datenschutzabteilung zuständig. Der Bereich AIS übernimmt in bestimmten Fällen die Bereitstellung der Testdaten z.B. bei der Durchführung von Testdatenanonymisierung im Fall der Nutzung von produktiven Daten zu Testzwecken.

Wichtig: Vor jeder Nutzung von produktiven Daten zu Testzwecken ist die Freigabe durch die Datenschutzabteilung einzuholen.

Für das InSprint Testing wird mit synthetischen Testdaten und falls erforderlich mit „gemockten“ Testdaten gearbeitet.

Die Datenstände auf den Testumgebungen sollten wenn möglich die produktionsrelevanten Datenkonstellationen abbilden. Anonymisierte und/oder pseudonymisierte Testdaten können bei Bedarf genutzt werden.

Der Umgebungsmanager sorgt in Kooperation mit dem Testmanager / Testdatenmanager und ggf. dem Testkoordinator dafür, dass die richtigen Testdaten zum richtigen Zeitpunkt für Tests zur Verfügung stehen. Die Definition, Generierung, Beschaffung und Bereitstellung der Testdaten wird im Testumgebungskonzeptes spezifiziert, welches der Umgebungsmanager verantwortet.

6 Testrealisierung und -durchführung

6.1 Testdurchführung und Testabdeckung

Für die Planung der Testdurchführung sind die durchzuführenden Testarten pro Teststufen mit der jeweils geplanten Testabdeckung für die am Test beteiligten Instanzen zu definieren und den Ansatz für die Durchführung von Re-Tests und Regressionstests festzulegen.

Aus den Beschreibungen sollte auch die Reihenfolge der durchzuführenden Aktivitäten sowie die Art und der Umfang der Testautomation (sofern diese geplant bzw. realisiert ist) hervorgehen.

Aufgrund der Tatsache, dass es sich hier um ein high-level-Testkonzept handelt, wird für die genaue Realisierungsbeschreibung sowie Testabdeckung auf den nachgelagerten und im weiteren Projektfortschritt zu erstellendem Release-Testplan je Release verwiesen.

6.2 Testautomatisierung

Ziel der Testautomatisierung ist die Senkung der Testkosten sowie die Verkürzung der Testdauer insbesondere bei Regressions- und Smoketests. Grundsätzlich wird ein hoher Grad an Automatisierung und Wiederverwendbarkeit der Testfälle angestrebt (siehe Kapitel 4.1).

Im Rahmen der DevOps Vorgehensweise ist die automatisierte Testfalldurchführung ein wichtiger Bestandteil z.B. im Sinne eines Conditional Deployments. Die automatisierten Tests werden zeitgenau von einem Continuous Integration Server gesteuert, welcher auf die Source-Verwaltung zugreift, ein Build erstellt und dies auf die gewünschten Umgebungen verteilt. Dort werden automatisierte Regressions- und Smoketests automatisch durchgeführt und durch Reports ein schnelles und präzises Feedback ermöglicht. Ein erfolgreiches Artefakt wird dann automatisch auf die gewünschte Umgebung deployed.

Die Grenzen der Automatisierung ergeben sich aus der technischen Machbarkeit und der Kostenbetrachtung und damit der Wirtschaftlichkeit bei der Definition, Automatisierung und Ausführung von automatisierten Testfällen. Es kann sinnvoll sein, technische Prüfungen zu automatisieren, wenn diese von Menschen fehleranfälliger oder nur näherungsweise durchgeführt werden können.

Die Automatisierung und Wartung von automatisierten Testfällen verursacht Aufwände, die sich bei regelmäßiger und häufiger Durchführung derselben Testportfolien bezahlt machen. In diesem Zusammenhang ist ein gutes Testportfoliomanagement wichtig, sodass zum einen die richtigen Testfälle risikobasiert ausgewählt werden und zum anderen die laufende Aktualisierung sichergestellt ist.

Die Arbeiten und Ziele der Testautomatisierung sind im Release-Testkonzept und Testplan zu definieren. Bei Bedarf kann zusätzlich ein Testautomatisierungskonzept erstellt werden.

Häufig werden die wesentlichen funktionalen Tests in frühen Teststufen definiert, die für die Verwendung in Smoke- und Regressionstests sinnvollerweise zu automatisieren sind. Als erste Näherung wurden zunächst die Smoke- und Regressionstests als am geeignetsten angesehen.

Selenium ist das vorgesehene Tool für die Automatisierung der Tests. Bei Bedarf kann ergänzend auf weitere Tools zurückgegriffen werden.

7 Testauswertung und -bericht

In diesem Kapitel wird dargestellt, welche Dokumente erzeugt werden sollen (z.B. Teststufenplan, Testfallbeschreibungen, Abweichungsbeschreibungen, wöchentlicher Statusbericht und Abschlussbericht) und welche die formalen und inhaltlichen Vorgaben für diese Dokumente sind.

7.1 Testdokumentation

Sämtliche Testdokumentation (Teststufenpläne, Testpläne, logische Testfälle, Testfälle, Testergebnisse und Fehler) erfolgt über den Azure DevOps. Dies betrifft auch Ergebnisse, die außerhalb des Azure DevOps entstanden sind (bspw. Last- und Performanceergebnisse), diese können entsprechend für einzelnen Testpläne hochgeladen werden.

Sollten noch zusätzliche Dokumente benötigt werden, (Testkonzept, Testspezifikation, Testfortschritt, Testprotokolle, Testberichte, Testabschlussbericht usw.), so werden diese wenn möglich unter Auswertung der Dokumentation aus Azure DevOps erstellt. Auf dem VITA SharePoint werden zumindest die Testkonzepte und Testabschlussberichte abgelegt, weitere Dokumente falls erforderlich.

7.2 Testmetriken

In Azure DevOps werden Bearbeitungs- und Statusinformationen erfasst, die für die Kalkulation von verschiedenen Metriken verwendet werden. Beispiele für Metriken: „Anzahl Fehler/Zeiteinheit (Woche/Sprint/Release)“, Clusterung von Fehlern nach Kritikalität, „Dauer der Fehlerbehebung“, etc.

Die Festlegung, welche Metriken genutzt werden, erfolgt im Rahmen des Release-Testkonzepts.

7.3 Reporting

In Azure DevOps werden Dashboards verwendet, um ein Echtzeit-Status-Update in Bezug auf den Testfortschritt, die Testergebnisse und das Fehlermanagement zu liefern. Aus diesen können aktuelle Auswertungen und Grafiken über die Testaktivitäten generiert werden.

Der Testmanager informiert alle relevanten Stakeholder in regelmäßigen Meetings (Projektstatusmeeting oder SCRUM of SCRUMs) über den Teststatus und -fortschritt.

Für die Projektleiter, Product Owner und Scrum Master sind die Dashboards jederzeit als Informationsquelle zugänglich. Bei Bedarf kann wöchentlich oder auf Sprintbasis ein Entwicklungs-/Teststatusreport über eine automatisierte Abfrage bereitgestellt werden.

Zum Abschluss jeder Teststufe erstellt der Testmanager einen Testabschlussbericht.

8 Testabschluss

8.1 Retrospektive

Als agile Praktik werden in den InSprint-Teams nach jedem Sprint im Rahmen der Retrospektive Prozesse, Werkzeuge, Fähigkeiten, Beziehungen, Herausforderungen und Erfahrungen reflektiert und soweit im nächsten Sprint umsetzbar als unmittelbar wirksame Verbesserung übernommen.

8.2 Lessons Learned

Im Rahmen von Lessons Learned-Veranstaltungen werden Erfahrungen aus der vorhergehenden Teststufe geteilt, bewertet und Verbesserungsvorschläge erarbeitet.

8.3 Optimierung des Testportfolios

Das Ende einer Phase kann auch als Anlass genutzt werden, das Testfall Portfolio zu prüfen:

Am Ende einer jeden Teststufe (InSprint-Test, SIT und UAT) werden zum Beispiel die Testfälle daraufhin überprüft, wie viele Fehler sie aufgedeckt haben, um zu überlegen, ob sie auch zukünftig regelmäßig durchgeführt werden sollen.

Diese Analyseergebnisse und Überlegungen fließen in die Zusammenstellung, Planung und ggf. (Um-)Priorisierung der Testfälle ein.

9 Anhang

9.1 Zu erstellende Artefakte

1. Release-Testkonzept
2. Testplan (enthält Meilensteine und Teststufenplan)
3. Testausführungsplan (zumeist in Azure DevOps geplant)
4. Testumgebungskonzept (enthält Testdatenkonzept und Berechtigungskonzept)
5. Testautomatisierungsstrategie und Testautomatisierungsarchitektur
6. Meilensteinplan Testing
7. Abgrenzung: Das CR-Verfahren ist als Teil der Projektgovernance seitens des Programms oder Projektes VITA zu definieren.

9.2 Tabellen- und Abbildungsverzeichnis

9.2.1 Tabellenverzeichnis

Tabelle 1: Quality Gates	8
Tabelle 2: Testfall Priorisierungsmetrik	9
Tabelle 3 - Bugs, Priorisierung & Klassifikation.....	9
Tabelle 4 - Priorität Bugs.....	10
Tabelle 5 : Deliverables per Phase	14
Tabelle 6: Rollen im Projekt 2	15
Tabelle 7: Umgebungsübersicht / technische Systemlandschaft.....	22
Tabelle 8: Übersicht Anwendungszwecke der unterschiedlichen Instanzen	23
Tabelle 9: Testziele pro Umgebung	23
Tabelle 10: Eingangskriterien.....	26
Tabelle 11 Testende / Abnahme Matrix	27
Tabelle 12 : Ablauf Implementierung User Story.....	31
Tabelle 13 Glossar	40
Tabelle 14: Dokumente und Links.....	41
Tabelle 15: Versionshistorie	42
Tabelle 16: Dokument-Informationen	42

Abbildungsverzeichnis

Abbildung 1: Defect Workflow Dokumentation (Scrum Template) aus Azure DevOps LINK	8
Abbildung 2: Grober Zeitplan für die Implementierung (Stand April 2021)	11
Abbildung 3: MVP Timeline (Stand April 2021)	12
Abbildung 4 Initial vermutete Aufwandsverteilung der Testrollen, die im Laufe der Releases ggf. angepasst wird (Stand 16.03.2021)	16
Abbildung 5: Testworkflow in InSprint Stage.....	30
Abbildung 6: Userstory	30
Abbildung 7: Test Workflow ab SIT	31

9.3 Glossar

Aus Verständnisgründen müssen sämtliche im Dokument benutzten Abkürzungen hier aufgeführt und bei Bedarf erläutert werden.

CR	Change Request
White-Box Test	Herleitung von Testfällen über Programmstruktur. Die Umsetzung der Anforderung wird dabei nicht geprüft
Black-Box Test	Fokussieren die Benutzeranforderungen und ignorieren die interne Programmstruktur. Die Testfälle werden aus der Spezifikation bzw. den Anforderungen an das Testobjekt abgeleitet
Funktionale Test	Zielen auf die Überprüfung der reinen Funktionalität des Testobjektes ab.
Nicht funktionale Tests NFT	Ziele auf die Überprüfung allgemeiner Qualitätsmerkmale von Software, außer der reinen Funktionalität. Diese sind entlang der Qualitätsmerkmale in der DIN ISO 25010 definiert
Re-Tests	Auch änderungsbezogene Test genannt. Jene Tests, die durch die Änderung einer Komponente oder eines Systems veranlasst werden. Hier wird nach Fehlernachtet und Regressionstest unterschieden.
Datenwörterbuch	eine unabhängige Darstellung von Metadaten, die zu Grunde liegende Datenstrukturen und die dazugehörigen Attribute beschreibt. Ein Datenwörterbuch wird mit Hilfe von Unternehmensvokabular aufgebaut.

Tabelle 13 Glossar

9.4 Referenzen

Hier die referenzierten Dokumente und andere Quellen einfügen.

Name	Link
Projekt SharePoint	https://adaccloud.sharepoint.com/:w:/r/teams/PRAR01328/Shared Documents/01_Analysephase/Test/Testkonzept/Testkonzept_VES20_Vers091.docx?d=wafafb11dce1d405eb2b25db605668c1e&csf=1&web=1&e=fKFUGv
Application Life-cycle Management (ALM)	http://intranet/Images/AIS Prozesse - ALM_tcm28-444479.pdf
Quality Gate Vorlage	http://intranet/Images/QualityGate_tcm28-425047.docx
Fehlermanagement Vorlage	https://connect.adac.de/bereich/00094/_layouts/15/WopiFrame2.aspx?sourcedoc=/bereich/00094/Dokumente/Fehlermanagement%20im%20ADAC%20V1.1.docx&action=default

Risiko-analyse Vorlage	https://adac-cloud.sharepoint.com/:x:/r/teams/PRAR01328/_layouts/15/doc2.aspx?sourcedoc=%7B89792B6C-D4B5-4A3F-B77F-2F85B081D012%7D&file=Vorlage_Risikoanalyse_DE_einfach.xls&action=default&mobileredirect=true&wdLOR=c0980FD6D-BB85-4CD8-9A2B-158BC21810B7&cid=bf9ac2ef-d549-4a88-a136-d438ebfdaa57		
Change Management Prozesse	TBD		
Testabschlussbericht Vorlage	http://intranet.adac.biz/Images/Testabschlussbericht_tcm28-425066.docx?Web=1		
Projekt Log	https://adaccloud.sharepoint.com/:x:/r/teams/PRAR01328/Shared Documents/01_Analysephase/_Projektdokumente/ADAC-Log.xlsx?d=w8dff1591f46d4f4792b15dc407dbc05e&csf=1&web=1&e=bi3eTP	-	Projekt
LuP Testabschlussbericht	https://adac-cloud.sharepoint.com/:w:/r/teams/PRAR01328/_layouts/15/Doc.aspx?sourcedoc=%7B4DBA19F3-F79C-432D-ADC1-AECE62A75F46%7D&file=Vorgaben_Testumgebung_0.95.docx&action=default&mobileredirect=true&wdLOR=c0592DB59-2CEC-48D9-AF76-4B2324F50ED0&cid=3ae00e59-d7f7-4f66-a1b9-0dbcf9695e1b		
ISTQB Glossar	ISTQB®GTB-Standardglossar-der-Testbegriffe-Deutsch-Englisch.pdf (german-testing-board.info)		

Tabelle 14: Dokumente und Links

9.5 Versions-Historie

Die **Versionsnummern** werden sequenziell hochgezählt. An Hand der Versionsnummer wird festgelegt, ob sich ein Dokument in Bearbeitung befindet oder freigegeben wurde. So bedeutet z.B. x.0, dass das **Dokument freigegeben** wurde. Eine Version x.y mit y > 0 bedeutet, dass sich das Dokument **in Bearbeitung** befindet.

Falls von einem Dokument in mehreren kleinen Schritten häufiger neue Versionen bis zur Freigabe erstellt werden, können für die Versionsnummer auch mehrere Nachkommastellen benutzt werden.

Version	erstellt am	durch	Änderungen
Version 0.1	22.02.2021	Barbara Paulini	Initiale
Version 0.2	05.03.2021	Jan Dijs	Review & Kommentare
Version 0.21	07.03.2021	Niels de Vos	Updates & Ergänzungen
Version 0.3	19.03.2021	Barbara Paulini	Updates & Ergänzungen
Version 0.5	26.03.2021	Barbara Paulini	Updates & Ergänzungen
Version 0.6	28.03.2021	Karin Boden	Review Karin Boden
Version 0.9	01.04.2021	Sebastian Seitz, Angela Giorgio-Marrano	Review Sebastian Seitz, Angela Giorgio-Marrano, Maximilian Bumiller
Version 0.926	20.04.2021	Karin Boden	Review, Kommentierung und Erarbeitung von Vorschlägen für die Überarbeitung insbesondere ab Kapitel 3.14

Version 095	23.04.2021	Sebastian Seitz	Review, Einarbeitung und Zusammenführung
Version 097	27.04.2021	Sebastian Seitz, Angela Giorgio-Marrano, Viara Velez, Astrid Stangenberg-Thurner	Review, Einarbeitung und Zusammenführung
Version 0.9.8	26.06.2021	Karin Boden	Review, Feedback und Ergänzungen
Version 0.99	01.07.2021	Sebastian Seitz, Martina Lypp	Finalisierung, Einarbeitung Feedback & Ergänzungen
Version 1.00	08.07.2021	Sebastian Seitz	Einarbeitung Anmerkungen Astrid Stangenberg

Tabelle 15: Versionshistorie

9.6 Abnahme des Dokuments

Ein Review des Dokumentes erfolgt durch:

- Alle VBO-Bereiche,
- Testmanager, vertreten durch
- Projektleitung, vertreten durch Martina Lypp, Josef Ringsgwandl, Karin Boden
- Der Fachbereich VBH, vertreten durch Hr. Seitz
- Die IT-Qualitätssicherung, vertreten durch Testmanagement des ADAC (XX XXX)

Die Abnahme des Dokuments erfolgt durch:

- Testmanagement, vertreten durch XX XXX
- Projektleitung, vertreten durch Martina Lypp, Josef Ringsgwandl, Karin Boden

9.7 Informationen zum Dokument

Den **Autor**, die **Versionsnummer** und das **Datum der letzten Änderung** bitte nur über den Menüeintrag **Datei->Eigenschaften->„Anpassen“** verändern, da es sonst immer wieder die typischen Word-Probleme entstehen, dass Feldinhalte automatisch geändert werden. **Titel und Thema** wie gewohnt über **Datei->Eigenschaften->„Datei-Info“** angeben.

Für ganz spezielle Kennzeichnungen eines Dokuments (z.B. „Vertraulich“, „Entwurf“) ist die Wasserzeichen-Technik (siehe Hilfethemen Microsoft Word) anzuwenden.

Datei:	
Share (X:):	
Autor(en):	
Version:	
vom:	
letzte Speicherung	
Freigabe durch:	Martina Lypp, Josef Ringsgwandl, Karin Boden
freigegeben am	Mai 2021 per eMail

Tabelle 16: Dokument-Informationen